

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN



HCMUTE

BÁO CÁO
MÔN HỌC: LẬP TRÌNH HỆ THỐNG

XÂY DỰNG MÔ HÌNH CHỒNG GIAO THỨC TCP/IP ĐƠN GIẢN

GVHD:	ThS. Đinh Công Doan
Nhóm SVTH:	Nhóm 5
Nguyễn Thành Được	22162011
Trương Nguyễn Anh Thư	22162047
Bùi Lê Thủy Tiên	22162048

Tp. Hồ Chí Minh, tháng 4 năm 2025

BẢNG PHÂN CÔNG NHIỆM VỤ

STT	HỌ TÊN - MSSV	NHIỆM VỤ	KÝ TÊN	ĐIỂM SỐ
1	Bùi Lê Thủy Tiên – 22162048			
2	Trương Nguyễn Anh Thư – 22162047			
3	Nguyễn Thành Được – 22162011			

Nhận xét của giáo viên:

.....

.....

.....

.....

.....

.....

.....

.....

KÝ TÊN

MỤC LỤC

PHẦN MỞ ĐẦU.....	1
1. Tóm tắt.....	1
2. Đặt vấn đề.....	1
2.1. Tóm lược những nghiên cứu trong và ngoài nước liên quan đến đề tài..	1
2.2. Tính cấp thiết cần nghiên cứu của đề tài	2
2.3. Một số tài liệu liên quan.....	3
2.4. Lý do chọn đề tài.....	3
2.5. Mục tiêu đề tài	4
2.6. Đối tượng và phạm vi nghiên cứu	5
2.7. Phương pháp nghiên cứu.....	5
PHẦN NỘI DUNG.....	6
CHƯƠNG 1: GIỚI THIỆU.....	6
CHƯƠNG 2: PHÂN TÍCH YÊU CẦU	7
2.1. Yêu cầu chung	7
2.2. Các thành phần chính của hệ thống	7
2.2.1. Tầng IP (Internet Protocol Layer).....	8
2.2.2. Tầng TCP (Transmission Control Protocol Layer).....	8
2.2.3. HTTP/HTTPS Server đơn giản (Lightweight HTTP/HTTPS Server).....	8
CHƯƠNG 3: MỘT SỐ KIẾN THỨC VỀ CHỖNG GIAO THỨC TCP/IP.....	10
3.1. TCP/IP là gì? (phần này cần sửa lại).....	10
3.2. Mô hình phân tầng trong TCP/IP	11
3.2.1. Tầng Vật lý (Physical Layer)	11

3.2.2.	Tầng Liên kết dữ liệu (Data Link Layer).....	11
3.2.3.	Tầng Mạng (Internet Layer).....	12
3.2.4.	Tầng Vận chuyển (Transport Layer).....	13
3.2.5.	Tầng Ứng dụng (Application Layer).....	14
3.3.	Đóng gói và mở gói dữ liệu.....	14
3.3.1.	Đóng gói.....	15
3.3.2.	Mở gói.....	16
3.4.	Một số giao thức và dịch vụ được triển khai trong mô hình TCP/IP.....	17
3.4.1.	IP – Internet Protocol.....	17
3.4.2.	TCP - Transmission Control Protocol.....	18
3.4.3.	HTTP – HyperText Transfer Protocol.....	18
3.4.4.	SSL – Secure Socket Layer.....	19
3.4.5.	HTTPS – HyperText Transfer Protocol Secure.....	19
CHƯƠNG 4:	ỨNG DỤNG.....	20
4.1.	Phương pháp kiểm thử (Thư đọc và viết lại).....	20
4.2.	Single-Threaded Server.....	21
4.2.1.	Mô tả ứng dụng và hạn chế.....	21
4.2.2.	Kiến trúc và các hàm cốt lõi.....	22
4.2.3.	Các bước xây dựng ứng dụng	27
4.2.4.	Kết luận.....	32
4.3.	Multi-Threaded Server (Multi-client with threading).....	33
4.3.1.	Mô tả ứng dụng và hạn chế.....	33
4.3.2.	Kiến trúc và các hàm cốt lõi.....	35

4.3.3. Các bước xây dựng ứng dụng	37
4.3.4. Kết luận.....	40
4.4. HTTP Server.....	41
4.4.1. Mô tả ứng dụng và hạn chế.....	41
4.4.2. Luồng hoạt động.....	42
4.4.3. Hạn chế chính của ứng dụng:.....	43
4.4.4. Kiến trúc và các hàm cốt lõi.....	45
4.4.5. Các bước xây dựng ứng dụng	46
4.4.6. Kết luận.....	51
4.5. HTTPS Server.....	51
4.5.1. Mô tả ứng dụng và hạn chế.....	51
4.5.2. Kiến trúc và các hàm cốt lõi.....	53
4.5.3. Các bước xây dựng ứng dụng	56
4.5.4. Kết luận.....	59
PHẦN KẾT LUẬN.....	61
1. Kết quả đạt được	61
2. Ưu điểm của đề tài.....	62
3. Nhược điểm và hạn chế.....	62
4. Hướng phát triển của đề tài.....	62
5. Tài liệu tham khảo	64

PHẦN MỞ ĐẦU

1. Tóm tắt

Trong bối cảnh Internet ngày càng trở nên phổ biến và đóng vai trò cốt lõi trong các hệ thống phần mềm hiện đại, việc hiểu và làm chủ các giao thức mạng trở thành yêu cầu thiết yếu đối với kỹ sư phần mềm và nhà nghiên cứu. Một trong những thành phần quan trọng nhất trong hệ thống mạng là chồng giao thức TCP/IP, đóng vai trò là nền tảng cho hầu hết các giao tiếp mạng trên toàn cầu.

Mặc dù có nhiều thư viện mạng và framework hỗ trợ sẵn, nhưng việc tự tay xây dựng một mô hình TCP/IP cơ bản giúp hiểu sâu sắc hơn về cách thức hoạt động nội tại của các tầng giao thức, cũng như cách dữ liệu được đóng gói, truyền tải và xử lý giữa các thiết bị.

Từ nhu cầu đó, đề tài lựa chọn hướng tiếp cận từ nền tảng, cụ thể là xây dựng một chồng giao thức TCP/IP đơn giản và một máy chủ HTTP tối giản (lightweight Web Server) hoạt động dựa trên mô hình đó. Thay vì sử dụng các hàm mạng hệ thống, toàn bộ cơ chế truyền thông sẽ được hiện thực thủ công bằng ngôn ngữ lập trình C – qua đó thể hiện trọn vẹn cơ chế vận hành của các tầng IP và TCP.

2. Đặt vấn đề

2.1. Tóm lược những nghiên cứu trong và ngoài nước liên quan đến đề tài

Hiện nay, có nhiều nghiên cứu và dự án liên quan đến triển khai chồng giao thức TCP/IP đơn giản nhằm phục vụ các hệ thống nhúng, mô phỏng hoặc giáo dục. Một số thư viện mã nguồn mở như lwIP (Lightweight IP) hay μ IP (Micro IP) đã được phát triển để cung cấp các giải pháp TCP/IP nhẹ, tối ưu cho thiết bị nhúng. Các nghiên cứu này không chỉ mang tính học thuật mà còn có giá trị thực tiễn, hỗ trợ phát triển các hệ thống mạng đơn giản nhưng hiệu quả.

Ngoài ra, nhiều nghiên cứu học thuật cũng đã đề cập đến việc mô phỏng giao thức mạng để giảng dạy và kiểm thử trong môi trường giả lập.

2.2. Tính cấp thiết cần nghiên cứu của đề tài

Việc nghiên cứu và triển khai một mô hình chồng giao thức TCP/IP đơn giản có ý nghĩa quan trọng vì các lý do sau:

Hiểu sâu về giao thức mạng: Việc tự xây dựng một TCP/IP Stack giúp người học và các nhà nghiên cứu hiểu rõ hơn về nguyên lý hoạt động của các giao thức cơ bản, thay vì chỉ sử dụng các API có sẵn trong hệ điều hành. Điều này giúp họ có cái nhìn toàn diện và sâu sắc hơn về cách thức các hệ thống mạng giao tiếp với nhau.

Giáo dục và đào tạo: Chương trình học về lập trình mạng sẽ trở nên phong phú và thực tiễn hơn khi sinh viên có thể tham gia vào quá trình triển khai một mô hình TCP/IP thực tế, không chỉ học lý thuyết mà còn thực hành trực tiếp qua các mô phỏng và kiểm thử.

Mô phỏng và kiểm thử: Việc triển khai TCP/IP Stack đơn giản cho phép tạo ra các môi trường giả lập để thử nghiệm và phân tích các tình huống mạng mà không cần phải phụ thuộc vào các hệ thống mạng thực tế. Điều này đặc biệt hữu ích trong các nghiên cứu mô phỏng hoặc kiểm thử bảo mật.

Hỗ trợ phát triển hệ thống nhúng: Các thiết bị nhúng có tài nguyên hạn chế thường không thể sử dụng các TCP/IP Stack phức tạp. Do đó, việc phát triển một mô hình nhẹ và có thể tùy chỉnh sẽ giúp tối ưu hóa tài nguyên của hệ thống.

Bảo mật mạng: Hiểu rõ cách thức hoạt động của các giao thức mạng không chỉ giúp phát hiện và khắc phục các lỗi trong quá trình giao tiếp mà còn hỗ trợ phát triển các biện pháp bảo mật mạnh mẽ hơn cho hệ thống mạng.

Bên cạnh những lợi ích về giáo dục và nghiên cứu, việc triển khai một mô hình TCP/IP đơn giản còn có ý nghĩa quan trọng trong việc phát triển các giải pháp mạng cho các hệ thống nhúng và các ứng dụng yêu cầu tối ưu hóa tài nguyên. Những kết quả và kinh nghiệm thu được từ dự án này có thể được áp dụng rộng rãi trong ngành công nghiệp phần mềm và các lĩnh vực nghiên cứu về mạng.

2.3. Một số tài liệu liên quan

1. Lucas van der Ploeg, [*Small TCP/IP stacks for micro controllers*](#).
2. Nilesh Rajbharti Microchip Technology Inc, [*The Microchip TCP/IP Stack Application Note*](#).

2.4. Lý do chọn đề tài

Quyết định nghiên cứu về việc triển khai một hệ thống máy chủ HTTP nhỏ xuất phát từ nhiều yếu tố thuyết phục:

Tính ứng dụng thực tiễn: Máy chủ HTTP là nền tảng của hạ tầng web hiện đại. Việc phát triển một hệ thống như vậy mang lại trải nghiệm thực tế với các công nghệ cốt lõi của internet, giúp hiểu rõ kiến trúc hệ thống web trong thế giới thực.

Giá trị giáo dục: Chủ đề này đòi hỏi sự hiểu biết sâu sắc về giao thức HTTP, lập trình mạng, mô hình xử lý đồng thời và các nguyên tắc thiết kế hệ thống, tạo ra một trải nghiệm học tập toàn diện kết hợp giữa lý thuyết và thực hành.

Khả năng tùy chỉnh: Việc triển khai một máy chủ HTTP quy mô nhỏ cho phép linh hoạt điều chỉnh và tùy biến để đáp ứng các yêu cầu cụ thể, đặc biệt trong những môi trường có tài nguyên hạn chế mà các giải pháp hiện có khó đáp ứng.

Yêu cầu tài nguyên hợp lý: Dự án có thể được thực hiện với các tài nguyên tính toán ở mức vừa phải nhưng vẫn tạo ra kết quả có ý nghĩa và có thể mở rộng, phù hợp với bối cảnh nghiên cứu học thuật.

Học tập liên ngành: Quá trình triển khai yêu cầu kiến thức từ nhiều lĩnh vực như mạng máy tính, bảo mật, tối ưu hóa hiệu suất và thiết kế phần mềm, giúp nâng cao hiểu biết toàn diện về hệ thống máy tính.

Tiềm năng phát triển: Chủ đề này mở ra nhiều hướng nghiên cứu mở rộng trong tương lai, chẳng hạn như tối ưu hóa hiệu suất, nâng cao bảo mật hoặc hỗ trợ các công nghệ và giao thức web mới.

Tính liên quan đến ngành công nghiệp: Kỹ năng hiểu và triển khai các hệ thống web backend có giá trị cao trong ngành phần mềm, giúp nghiên cứu này có tính ứng dụng trực tiếp đối với sự phát triển nghề nghiệp.

Cải thiện kỹ năng giải quyết vấn đề: Các thách thức kỹ thuật trong việc triển khai máy chủ giúp rèn luyện tư duy phản biện và khả năng giải quyết vấn đề nâng cao, có thể áp dụng cho nhiều lĩnh vực kỹ thuật phức tạp khác.

Đóng góp cho cộng đồng mã nguồn mở: Sản phẩm của nghiên cứu có thể trở thành một giải pháp nhẹ, góp phần vào cộng đồng mã nguồn mở và phục vụ các trường hợp sử dụng cụ thể mà các giải pháp hiện tại còn quá phức tạp.

2.5. Mục tiêu đề tài

Thiết kế kiến trúc TCP/IP đơn giản: Xây dựng một chồng giao thức TCP/IP đơn giản với các tầng giao thức cơ bản, bao gồm tầng IP và tầng TCP. Hệ thống sẽ hỗ trợ các chức năng cơ bản của TCP/IP, như việc đóng gói và phân phối gói tin, thiết lập kết nối, kiểm soát luồng dữ liệu và xử lý lỗi.

Triển khai giao thức TCP và IP: Cài đặt và kiểm thử các chức năng cơ bản của giao thức TCP và IP, bao gồm việc gửi và nhận gói tin, phân mảnh gói tin, thiết lập kết nối TCP (ba bước bắt tay), xác nhận gói tin qua cơ chế số thứ tự (Sequence Number) và xác nhận (ACK).

Xây dựng HTTP Server nhẹ: Tạo ra một máy chủ HTTP đơn giản, có thể xử lý các yêu cầu HTTP GET cơ bản từ client, và trả về nội dung tĩnh thông qua kết nối TCP.

Kiểm thử và đánh giá hệ thống: Đánh giá hiệu suất của mô hình TCP/IP Stack đơn giản thông qua việc chạy trong môi trường Docker và thử nghiệm với các yêu cầu HTTP. Các chỉ số hiệu suất như thời gian phản hồi, khả năng duy trì kết nối và tốc độ xử lý sẽ được kiểm tra.

Tối ưu hóa hiệu suất: Tối ưu hóa mã nguồn và cấu trúc bộ đệm trong hệ thống TCP/IP để giảm thiểu tài nguyên sử dụng, đảm bảo hệ thống có thể hoạt động hiệu quả trong môi trường tài nguyên hạn chế.

(cần thêm phần xây dựng HTTPS Server bảo mật)

2.6. Đối tượng và phạm vi nghiên cứu

- Đối tượng nghiên cứu: Mô hình giao tiếp TCP/IP đơn giản, tập trung vào việc triển khai tầng mạng (IP) và tầng giao vận (TCP) bằng ngôn ngữ C.
- Phạm vi nghiên cứu: Chỉ hỗ trợ IPv4, không triển khai IPv6. Tập trung vào TCP, không bao gồm UDP hoặc ICMP. Không tích hợp với hệ thống mạng thực tế, chỉ chạy trong môi trường giả lập.

2.7. Phương pháp nghiên cứu

Phương pháp phân tích lý thuyết: Phương pháp sẽ được sử dụng để nghiên cứu và tìm hiểu các nguyên lý hoạt động của các giao thức TCP và IP, cũng như cách thức hoạt động của HTTP. Các tài liệu tiêu chuẩn và RFC (Request for Comments) sẽ được tham khảo để nắm vững kiến thức về các giao thức mạng này.

Phương pháp thực nghiệm: Phương pháp sẽ bao gồm việc lập trình và triển khai mô hình TCP/IP Stack đơn giản, cùng với máy chủ HTTP. Các chức năng cơ bản của các giao thức TCP và IP sẽ được kiểm thử thông qua việc viết mã và thực hiện kiểm thử đơn vị. Máy chủ HTTP cũng sẽ được kiểm thử với các yêu cầu HTTP GET đơn giản để đảm bảo hoạt động đúng.

Phương pháp mô phỏng: Mô phỏng sẽ được thực hiện trong môi trường Docker, nơi các hệ thống mạng được tái hiện và kiểm thử. Docker sẽ giúp tạo ra các container để mô phỏng các hệ thống và kết nối mạng giữa chúng, trong khi đó việc xử lý và phản hồi HTTP sẽ được kiểm tra và đánh giá trong môi trường này.

PHẦN NỘI DUNG

CHƯƠNG 1: GIỚI THIỆU

Trong quá trình phát triển phần mềm, đặc biệt là các ứng dụng mạng, một trong những vấn đề quan trọng cần giải quyết là cách thức giao tiếp hiệu quả và đáng tin cậy giữa các thiết bị trong môi trường mạng. Bộ giao thức TCP/IP là nền tảng cốt lõi cho hầu hết các giao tiếp mạng hiện đại, từ duyệt web, gửi email đến các dịch vụ đám mây. Tuy nhiên, việc chỉ sử dụng các thư viện hoặc framework mạng cấp cao thường che giấu các chi tiết hoạt động bên trong của các giao thức này.

Để có được sự hiểu biết sâu sắc hơn về nguyên tắc hoạt động của các giao thức truyền thông cơ bản và cách xây dựng các ứng dụng mạng từ nền tảng, đề án này tập trung vào việc triển khai một chuỗi các mô hình máy chủ mạng bằng ngôn ngữ lập trình C, sử dụng giao diện lập trình socket cấp thấp. Thay vì xây dựng lại toàn bộ chồng giao thức TCP/IP từ đầu, hướng tiếp cận này tập trung vào việc tương tác trực tiếp với các dịch vụ do chồng giao thức của hệ điều hành cung cấp, qua đó tìm hiểu cách thức hoạt động của TCP/IP thông qua việc xây dựng và kiểm thử các ứng dụng thực tế.

Mục tiêu chính là tạo ra các phiên bản máy chủ từ đơn giản đến phức tạp hơn, bao gồm:

- Máy chủ TCP đơn luồng và đa luồng để hiểu cơ chế kết nối và xử lý đồng thời.
- Máy chủ HTTP/HTTPS nhẹ, có khả năng phục vụ các yêu cầu web cơ bản.

Quá trình này không chỉ giúp làm rõ các khái niệm lý thuyết về socket, kết nối, truyền dữ liệu mà còn đối mặt với các thách thức thực tế như quản lý tài nguyên, xử lý lỗi và tối ưu hóa hiệu suất (ví dụ, thông qua việc chuyển từ mô hình đơn luồng sang đa luồng).

Bên cạnh đó, một ứng dụng thực tiễn của việc tìm hiểu này là triển khai một Lightweight Web Server, hoạt động trên nền tảng TCP/IP đã được xây dựng thông qua

socket API. Máy chủ web này được thiết kế để có khả năng xử lý các yêu cầu đơn giản từ client, đảm bảo đáp ứng đủ dung lượng cần thiết mà không tiêu tốn quá nhiều tài nguyên hệ thống, phù hợp cho mục đích học tập, kiểm thử.

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU

2.1. Yêu cầu chung

Để hệ thống hoạt động đúng theo nguyên tắc của mô hình TCP/IP, cần đáp ứng một số tiêu chí kỹ thuật quan trọng. Trước hết, hệ thống phải hỗ trợ được giao tiếp mạng cơ bản, tức là có khả năng đóng gói và truyền tải dữ liệu đúng theo chuẩn TCP/IP, đảm bảo các gói tin được xử lý chính xác theo từng tầng giao thức.

Bên cạnh đó, việc ưu tiên sử dụng giao diện cấp thấp – giao diện lập trình socket chuẩn (ví dụ: POSIX sockets trong C) thay vì các thư viện hoặc framework mạng cấp cao hơn. Với mục tiêu tương tác trực tiếp hơn với chồng giao thức TCP/IP của hệ điều hành, hiểu rõ hơn về các tương tác mạng cơ bản và tránh sự trừu tượng hóa không cần thiết.

Về mặt kiến trúc, mô hình được thiết kế ứng dụng theo hướng module với khả năng mở rộng. Ví dụ, từ máy chủ TCP cơ bản có thể phát triển thành máy chủ xử lý đồng thời, rồi thành máy chủ HTTP, và có tiềm năng tích hợp các giao thức ứng dụng khác hoặc cơ chế xử lý khác trong tương lai. Điều này giúp hệ thống linh hoạt, phục vụ tốt hơn cho mục tiêu nghiên cứu và ứng dụng trong các lĩnh vực khác nhau.

Cuối cùng, một ứng dụng cụ thể được triển khai là máy chủ HTTP tối giản. Hệ thống này có khả năng gửi và nhận các yêu cầu HTTP cơ bản (GET) thông qua giao thức TCP tự xây dựng, minh chứng cho khả năng hoạt động thực tiễn của ngăn xếp giao thức đã phát triển.

2.2. Các thành phần chính của hệ thống

Hệ thống ứng dụng mạng được xây dựng sẽ tương tác với các tầng khác nhau của mô hình TCP/IP. Yêu cầu tập trung vào việc sử dụng đúng các dịch vụ mà các tầng này cung cấp thông qua Socket API và xây dựng logic ở tầng ứng dụng.

2.2.1. Tầng IP (Internet Protocol Layer)

Phạm vi dự án sẽ không trực tiếp triển khai tầng IP, nhưng hệ thống vẫn cần tuân thủ các quy tắc hoạt động tầng này thông qua cấu hình socket và sử dụng đúng địa chỉ IP khi khởi tạo kết nối. Việc định tuyến, phân mảnh, kiểm tra lỗi trong header IP... sẽ được hệ điều hành đảm nhiệm. Do đó, nhiệm vụ của ứng dụng là tạo ra các socket hoạt động ổn định trên nền IP, đảm bảo rằng mọi gói tin được gửi đúng đến địa chỉ mong muốn và có thể nhận phản hồi chính xác.

2.2.2. Tầng TCP (Transmission Control Protocol Layer)

Ứng dụng sẽ phải được sử dụng các hàm socket API để yêu cầu và quản lý các kết nối TCP/IP tin cậy.

Khi thiết lập kết nối chuẩn, trong đó phía Server sử dụng các lời gọi bind, listen, accept để mở cổng chờ kết nối, phía Client khởi tạo kết nối bằng lời gọi connect. Quá trình này kích hoạt cơ chế “bắt tay ba bước” (Three-way Handshake) do TCP của hệ điều hành tự thực hiện.

Khi kết nối được thiết lập, việc gửi và nhận dữ liệu giữa hai đầu được tiến hành thông qua các hàm read/recv và write/send, đảm bảo dữ liệu được truyền nhận theo luồng (stream – oriented) và duy trì thứ tự cũng như độ tin cậy. Đồng thời, hệ thống phải quản lý vòng đời của mỗi kết nối, từ thiết lập đến truyền dữ liệu cho đến đóng kết nối – bằng lời gọi close, kích hoạt quá trình “bắt tay bốn bước” (Four-way Handshake).

Một yêu cầu quan trọng khác là khả năng xử lý nhiều kết nối đồng thời ở phía Server. Điều này thường được hiện thực bằng cách tạo luồng mới (pthread) cho mỗi kết nối đến, giúp Server hoạt động đa luồng và phục vụ nhiều Client song song.

2.2.3. HTTP/HTTPS Server đơn giản (Lightweight HTTP/HTTPS Server)

Thành phần cốt lõi ở tầng Ứng dụng của hệ thống là một máy chủ Web nhẹ, được thiết kế để phục vụ cả giao thức HTTP (không mã hóa) và HTTPS (có mã hóa qua SSL/TLS). Mục tiêu của thành phần này là xử lý các yêu cầu từ Client, phân tích nội dung yêu cầu và gửi lại phản hồi phù hợp theo đúng định dạng HTTP hay HTTPS chuẩn.

Ở chế độ HTTP, sau khi thiết lập kết nối TCP thành công, Server sẽ tiếp nhận dữ liệu thô từ socket và phân tích nội dung để trích xuất các thông tin quan trọng như phương thức (GET, POST), đường dẫn tài nguyên (URL) và phiên bản giao thức HTTP. Với mỗi yêu cầu đến, hệ thống sẽ ánh xạ URI tới tệp tương ứng trong thư mục tài nguyên tĩnh (chẳng hạn như /www), kiểm tra sự tồn tại và quyền truy cập của tệp, sau đó phản hồi lại nội dung nếu tệp hợp lệ. Các loại có thể phục vụ bao gồm HTML, CSS, hình ảnh, văn bản... Nếu tệp không tồn tại hoặc không thể truy cập, Server sẽ trả về mã lỗi thích hợp như 404 Not Found.

Phần phản hồi HTTP được xây dựng thủ công với đầy đủ các thành phần: dòng trạng thái (ví dụ: “HTTP/1.1 200 OK”, “HTTP/1.1 404 Not Found”), các header như Content-Type, Content-Length và phần thân (Body) chứa nội dung thực tế. Việc này giúp đảm bảo Server có thể hoạt động độc lập mà không cần dùng các framework phức tạp.

Ở chế độ HTTPS, Server được tăng cường bảo mật thông qua tích hợp thư viện mã hóa như OpenSSL. Sau khi thiết lập kết nối TCP, Server sẽ tiến hành quá trình SSL/TLS Handshake để thương lượng thuật toán mã hóa và thiết lập phiên làm việc an toàn với Client. Toàn bộ dữ liệu trao đổi sau đó sẽ được mã hóa và giải mã bằng các hàm API chuyên biệt như SSL_read, SSL_write.

Để hỗ trợ HTTPS, hệ thống có khả năng tải lên chứng chỉ số (Certificate) và khóa riêng tư (Private Key), từ đó xác thực danh tính Server với Client. Đồng thời, Server cũng phải được cấu hình để lắng nghe trên một cổng chuyên biệt cho HTTPS, chẳng hạn như 443 hoặc cổng tùy chỉnh như 9443.

Tổng thể, dù sử dụng HTTP hay HTTPS, tất cả hoạt động truyền nhận đều diễn ra qua kết nối socket. Server cần sử dụng chính xác các hàm socket API ở tầng dưới, hoặc hàm được bao bọc bởi thư viện SSL/TLS trong trường hợp mã hóa, để thực hiện đầy đủ vòng đời của một phiên giao tiếp mạng ở tầng ứng dụng.

CHƯƠNG 3: MỘT SỐ KIẾN THỨC VỀ CHỖNG GIAO THỨC TCP/IP

3.1. TCP/IP là gì?

TCP/IP (Transmission Control Protocol/ Internet Protocol) là bộ giao thức sử dụng để kết nối các thiết bị và truyền tải thông tin trên Internet cũng như trong các hệ thống mạng máy tính. Nó xác định cách dữ liệu được đóng gói, truyền đi, định tuyến, nhận và xử lý để đảm bảo liên lạc hiệu quả giữa các thiết bị đầu cuối.

TCP/IP được phát triển vào những năm 1970 bởi Bộ Quốc phòng Mỹ (DoD) để phục vụ ARPANET – tiền thân của Internet ngày nay. Nhờ tính linh hoạt và khả năng mở rộng, TCP/IP đã trở thành tiêu chuẩn mạng phổ biến nhất, được sử dụng rộng rãi trên các hệ điều hành như Windows, Linux, macOS, thiết bị IoT, mạng doanh nghiệp và hạ tầng Internet toàn cầu.



Cấu trúc của TCP/IP gồm 5 tầng, được chồng lên nhau:

- Tầng Vật lý (Physical Layer): Truyền dữ liệu dạng tín hiệu điện, ánh sáng hoặc sóng vô tuyến qua phương tiện vật lý như cáp đồng, cáp quang, Wi-Fi.
- Tầng Liên kết Dữ liệu (Data Link Layer): Đóng gói dữ liệu thành frame, gán địa chỉ vật lý (MAC), kiểm soát truy cập đường truyền và phát hiện lỗi.

dữ liệu. Định khung Giao thức điểm-điểm (PPP) và định khung Ethernet IEEE 802.2 là hai ví dụ về các giao thức Tầng Liên kết Dữ liệu.

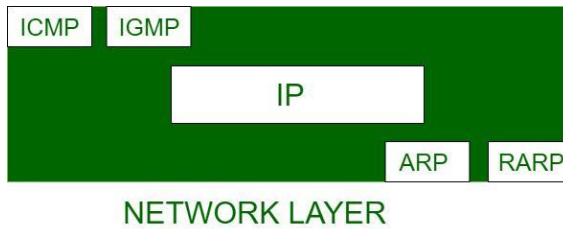
3.2.3. Tầng Mạng (Internet Layer)

Chịu trách nhiệm định tuyến các gói tin giữa các máy tính, quyết định sử dụng giao thức cấp thấp nào khi có nhiều giao diện (multiple interface) và xác định gửi gói tin trực tiếp đến máy chủ hoặc qua router. Khi gói tin lớn hơn kích thước hỗ trợ bởi phương tiện vật lý, tầng Mạng sẽ chia gói tin thành các gói tin nhỏ hơn, một quá trình được gọi là “phân mảnh và tái lắp ráp” (fragmentation and reassembly). Ngoài ra, tầng này cũng cung cấp một số dịch vụ kiểm soát gói tin để đảm bảo rằng chúng không bị gửi từ router này sang router khác một cách vô hạn.

Tuy nhiên, tầng Mạng không đảm bảo rằng gói tin sẽ đến nơi thành công. Sau khi gói tin rời khỏi máy tính gửi, tầng Mạng không theo dõi trạng thái của gói tin vì vậy nó không xác nhận được rằng gói tin đã được nhận thành công hay chưa.

Các giao thức chính nằm ở lớp này là:

- IP: là viết tắt của Giao thức Internet (Internet Protocol), nó chịu trách nhiệm phân phối các gói từ máy nguồn đến máy đích bằng cách xem địa chỉ IP trong tiêu đề gói. IP có 2 phiên bản: IPv4 và IPv6.
- ICMP: là viết tắt của Giao thức Tin nhắn điều khiển Internet (Internet Control Message Protocol). Nó được gói gọn trong các gói dữ liệu IP và chịu trách nhiệm cung cấp cho máy chủ thông tin về sự cố mạng.
- ARP: là viết tắt của Giao thức Phân giải địa chỉ (Address Resolution Protocol), một trong những giao thức quan trọng. Nó chịu trách nhiệm tìm kiếm địa chỉ phần cứng của máy chủ từ một địa chỉ IP đã biết. Một số loại ARP: Reverse ARP, Proxy ARP, Inverse ARP.



Hình 1: Các Giao Thức Hoạt Động ở Tầng Mạng

3.2.4. Tầng Vận chuyển (Transport Layer)

Tầng này gồm hai giao thức chính: Transmission Control Protocol (TCP) và User Datagram Protocol (UDP).

- Giao thức điều khiển truyền dẫn (TCP): đảm bảo dữ liệu được gửi từ các tầng cao hơn được truyền tải theo đúng thứ tự và không bị mất mát hay hư hỏng. Trước khi trao đổi dữ liệu, TCP phải thiết lập một kết nối giữa hai thiết bị thông qua quá trình bắt tay ba bước (Three-Way Handshake). Sau đó, dữ liệu được gửi và nhận dưới dạng luồng byte liên tục, mỗi byte có thể được đánh số thứ tự chính xác. Nếu không nhận được xác nhận (ACK) từ phía bên kia, hoặc nhận được xác nhận (ACK) cho dữ liệu đã gửi trước đó hai lần, TCP sẽ tự động gửi lại gói tin cho đến khi toàn bộ dữ liệu được xác nhận thành công. Đây là lý do tại sao TCP phải kết hợp với IP để đảm bảo truyền dữ liệu tin cậy trên mạng Internet.
- Giao thức gói dữ liệu người dùng (UDP): truyền và nhận dữ liệu dưới dạng các gói tin (datagram) mà không đảm bảo việc truyền tải. Nội dung dữ liệu có thể được gửi kèm hoặc không kèm kiểm tra tổng (checksum), tùy thuộc vào từng triển khai.

Tùy vào đặc điểm của từng giao thức mà người ta sử dụng chúng trong các tình huống khác nhau. Ví dụ, TCP thường được sử dụng trong các ứng dụng yêu cầu độ tin cậy cao như truyền tải file (FTP) hoặc gửi email (SMTP). Trong khi đó, UDP được ưu tiên trong các ứng dụng yêu cầu tốc độ và độ trễ thấp như phát trực tiếp video (live streaming) hoặc truyền dữ liệu thời gian thực VoIP).

3.2.5. Tầng Ứng dụng (Application Layer)

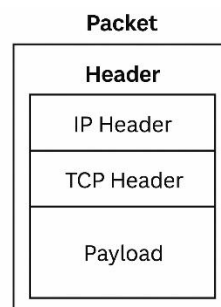
Chứa các ứng dụng chung và thư viện chức năng cho các ứng dụng sử dụng. Một số ứng dụng chạy trên TCP bao gồm Giao thức truyền tập tin (FTP), mô phỏng terminal từ xa (TELNET ở chế độ dòng lệnh, TN3270 ở chế độ toàn màn hình), Thư điện tử (SMTP) và Entire Net-Work. Một số ứng dụng chạy trên UDP bao gồm Máy chủ tập tin mạng (NFS) và Máy chủ tên miền (DNS).

Ngoài ra, tầng này còn cung cấp các thư viện chức năng để đơn giản hóa giao diện giữa ứng dụng và TCP/UDP của tầng Vận chuyển. Thư viện phổ biến nhất là Sockets, cho phép ứng dụng viết bằng ngôn ngữ C truy cập TCP như một thiết bị nhập/xuất dòng khác. Một thư viện khác ít phổ biến hơn là Remote Procedure Call (RPC), cho phép ứng dụng gọi các hàm nằm trong một ứng dụng khác trên máy tính khác.

Môi trường mà ứng dụng chạy thường quyết định giao diện được sử dụng giữa nó và TCP hoặc UDP. Ví dụ, hầu hết các ứng dụng trên UNIX, OS/2 và Windows sử dụng Sockets, trong khi các ứng dụng trên hệ thống IBM mainframe thường sử dụng RPC.

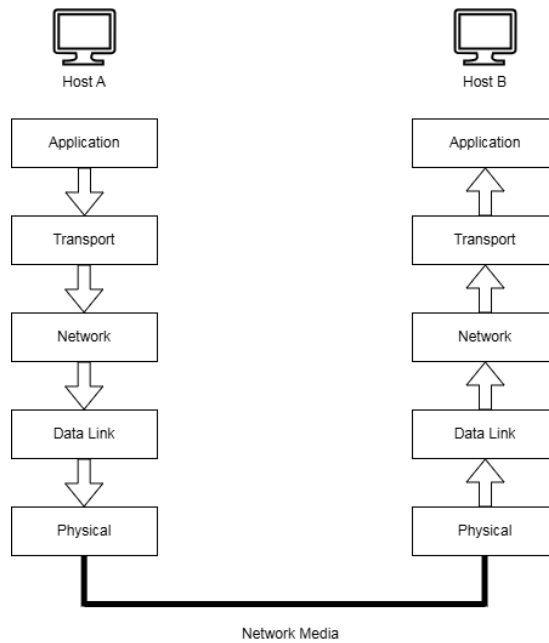
3.3. Đóng gói và mở gói dữ liệu

Gói tin là đơn vị dữ liệu cơ bản được truyền qua mạng. Mỗi gói tin bao gồm: thông tin điều khiển, định tuyến, kiểm tra lỗi và dữ liệu thực tế được truyền đi.



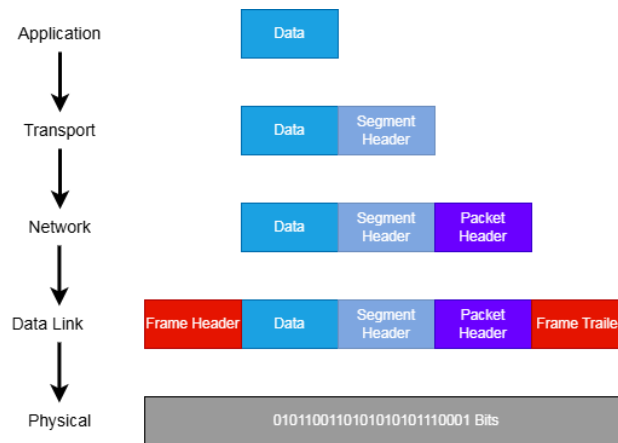
Hình 2: Cấu Trúc Gói Tin TCP/IP

Khi người dùng gửi một lệnh hoặc một thông điệp qua mạng, dữ liệu sẽ đi xuống qua các tầng giao thức tại máy gửi (host gửi) và đi lên qua các tầng tương ứng tại máy nhận (host nhận). Quá trình này được mô tả như sau:



Hình 3: Cách một gói tin di chuyển qua chồng giao thức TCP/IP

3.3.1. Đóng gói



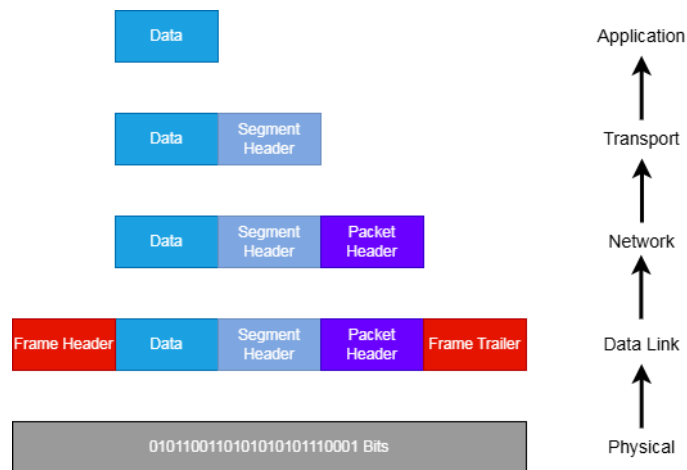
Hình 4: Quá Trình Đóng Gói Dữ Liệu Trong Mô Hình TCP/IP

- Tầng ứng dụng (Application Layer) – Người dùng bắt đầu giao tiếp: Dữ liệu được tạo ra bởi người dùng hoặc chương trình (ví dụ: trình duyệt web, email,...). Dữ liệu ở đây gọi là **Message (Thông điệp)**. Thông điệp này được chuyển xuống tầng dưới là **Transport Layer**.
- Tầng giao vận (Transport Layer) – Bắt đầu đóng gói dữ liệu: Thêm **tiêu đề (header)** vào để xác định **port gửi và nhận, thứ tự gói tin và trường kiểm**

tra lỗi (checksum) dùng để chuyển đúng dữ liệu đến ứng dụng đích. Nếu dùng **TCP**, dữ liệu được gọi là **TCP Segment**. Nếu dùng **UDP**, dữ liệu được gọi là **UDP Datagram**.

- Tầng Internet (Internet Layer) – Tạo Datagram IP: IP sẽ gắn thêm **header IP** chứa: địa chỉ IP nguồn và đích, độ dài gói tin, thứ tự nếu có phân mảnh. IP đảm bảo gói tin đến đúng máy nhận
- Tầng liên kết dữ liệu (Data Link Layer) – Tạo Frame: Giao thức tầng này như **Ethernet** hoặc **PPP** sẽ đóng gói Datagram IP thành **frame**. Gắn thêm header và footer để tạo thành **frame**, gồm: thông tin địa chỉ phần cứng (MAC), trường kiểm tra lỗi (**CRC**) để kiểm tra dữ liệu khi truyền đi.
- Tầng vật lý (Physical Layer) – Truyền gói tin: tầng vật lý nhận frame và chuyển đổi dữ liệu thành tín hiệu điện, ánh sáng hoặc sóng radio để truyền qua **mạng vật lý** (Network Media). Gói tin bắt đầu hành trình đến máy đích.

3.3.2. Mở gói



Hình 5: Quá Trình Mở Gói Dữ Liệu Trong Mô Hình TCP/IP

Khi dữ liệu đến máy đích, quá trình mở gói xảy ra theo chiều ngược lại (từ tầng thấp lên tầng cao). Mỗi tầng sẽ gỡ bỏ header/trailer của mình và chuyển phần còn lại lên tầng trên.

- Tầng Vật lý (Layer 1): Nhận dòng bit từ cáp mạng hoặc sóng, chuyển lên tầng Liên kết dữ liệu.

- Tầng Liên kết Dữ liệu (Layer 2): Kiểm tra và loại bỏ header/trailer, trích xuất IP Datagram và chuyển lên tầng Mạng.
- Tầng Mạng (Layer 3): Gỡ bỏ header IP để biết địa chỉ nguồn/đích IP, Trích xuất TCP Segment hoặc UDP Datagram và chuyển lên tầng Giao vận.
- Tầng Giao Vận (Layer 4): Gỡ bỏ header TCP/UDP để biết port đích, Dữ liệu còn lại chính là Message được chuyển lên tầng Ứng dụng.
- Tầng Ứng dụng (Layer 5): Ứng dụng nhận được đúng dữ liệu cần xử lý (ví dụ: hiển thị web, nhận email, v.v.).

3.4. Một số giao thức và dịch vụ được triển khai trong mô hình TCP/IP

3.4.1. IP – Internet Protocol

Internet Protocol (IP) là giao thức cốt lõi ở tầng mạng (Network Layer) trong mô hình TCP/IP. Nhiệm vụ chính của IP là định tuyến và truyền các gói tin (packets) từ nguồn đến đích trong một mạng hoặc qua nhiều mạng trung gian.

IP là giao thức không kết nối (Connectionless) và không đảm bảo độ tin cậy. Điều này có nghĩa là IP chỉ gửi các gói tin mà không theo dõi trạng thái kết nối hoặc kiểm tra xem gói có đến nơi hay không. Trách nhiệm đảm bảo độ tin cậy được giao cho các tầng trên như TCP.

Một số đặc điểm chính của IP:

- Định danh thiết bị qua địa chỉ IP: Mỗi thiết bị trong mạng có một địa chỉ IP duy nhất giúp định tuyến đúng đích.
- Chia dữ liệu thành các datagram: Mỗi gói IP (datagram) có phần tiêu đề chứa thông tin định tuyến và phần dữ liệu mang nội dung truyền tải.
- Định tuyến qua nhiều mạng: IP hỗ trợ truyền dữ liệu qua nhiều thiết bị trung gian (router) để đến đúng đích.

Hiện tại, IP có hai phiên bản:

- IPv4: Phiên bản phổ biến nhất, sử dụng địa chỉ 32-bit, nhưng đang dần cạn kiệt.

- IPv6: Phiên bản mới, sử dụng địa chỉ 128-bit để giải quyết vấn đề thiếu địa chỉ và tăng cường hiệu năng.

Nhờ IP, Internet có thể kết nối hàng tỷ thiết bị trên toàn cầu và cho phép dữ liệu được truyền đi hiệu quả giữa các hệ thống khác nhau.

3.4.2. TCP - Transmission Control Protocol

Giao thức điều khiển truyền dẫn (Transmission Control Protocol - TCP) là một trong những giao thức cốt lõi của mô hình TCP/IP, hoạt động ở tầng vận chuyển (Transport Layer). TCP cung cấp dịch vụ hướng kết nối (Connection-Oriented Service), đảm bảo rằng dữ liệu được truyền đi một cách tin cậy, có thứ tự và không bị mất gói.

Quá trình truyền dữ liệu qua TCP được thực hiện thông qua các bước sau:

- Bắt tay ba bước (Three-Way Handshake): Thiết lập kết nối giữa Client và Server trước khi truyền dữ liệu.
- Truyền dữ liệu: Dữ liệu được chia thành các gói tin, mỗi gói tin có số thứ tự và khi nhận được thì bên nhận sẽ gửi tính hiệu xác nhận (ACK).
- Kết thúc kết nối: Khi hoàn thành quá trình truyền dữ liệu, TCP thực hiện bắt tay bốn bước (Four-Way Handshake) để đóng kết nối.

Ngoài ra, giao thức TCP còn sử dụng cơ chế cửa sổ trượt (Sliding window) để đảm bảo bên gửi không gửi dữ liệu nhanh hơn khả năng xử lý của bên nhận, tránh làm tràn bộ đệm (buffer) của bên nhận.

Nhờ vào cơ chế kiểm soát lỗi và đảm bảo độ tin cậy, TCP thường được sử dụng cho các ứng dụng yêu cầu truyền dữ liệu chính xác như truyền file, email, duyệt web và các giao dịch ngân hàng trực tuyến.

3.4.3. HTTP – HyperText Transfer Protocol

HTTP (HyperText Transfer Protocol) là giao thức phổ biến được sử dụng để trao đổi dữ liệu giữa web client (trình duyệt) và Web Server. HTTP hoạt động trên tầng ứng dụng (Application Layer) của mô hình TCP/IP và sử dụng TCP làm giao thức truyền tải.

Một số đặc điểm chính của HTTP:

- Không trạng thái (Stateless): Mỗi yêu cầu HTTP được thực hiện độc lập, không lưu giữ thông tin về phiên làm việc trước đó.
- Sử dụng phương thức yêu cầu (Request Methods): Các phương thức phổ biến GET, POST, PUT, DELETE.
- Dữ liệu không được mã hóa: HTTP không cung cấp cơ chế bảo mật, khiến dữ liệu có thể bị chặn và đánh cắp.

Vì HTTP không đảm bảo an toàn, nó thường được sử dụng trong các trang web công khai, nơi không yêu cầu mã hóa dữ liệu.

3.4.4. SSL – Secure Socket Layer

SSL (Secure Socket Layer) là một giao thức mã hóa được sử dụng để bảo mật kết nối trên Internet. SSL hoạt động ở lớp giao vận (Transport Layer), giữa TCP và các giao thức ứng dụng như HTTP hoặc FTP. Mục tiêu chính của SSL là:

- Mã hóa dữ liệu (Encryption) Đảm bảo dữ liệu được bảo vệ trong quá trình truyền.
- Xác thực danh tính (Authentication): Đảm bảo Client và Server hợp lệ.
- Tính toàn vẹn (Integrity): Ngăn chặn dữ liệu bị sửa đổi trong quá trình truyền tải.

SSL thường được sử dụng để bảo vệ các giao dịch nhạy cảm như giao dịch ngân hàng, thanh toán trực tuyến và đăng nhập tài khoản. Hiện nay, SSL đã được thay thế bởi TLS (Transport Layer Security), một phiên bản nâng cấp với khả năng bảo mật tốt hơn.

3.4.5. HTTPS – HyperText Transfer Protocol Secure

HTTPS (HyperText Transfer Protocol Secure) là phiên bản bảo mật của HTTP, sử dụng SSL/TLS để mã hóa dữ liệu, giúp bảo vệ thông tin trong quá trình truyền tải.

Sự khác biệt chính giữa HTTPS và HTTP:

- HTTPS sử dụng SSL/TLS để mã hóa dữ liệu, đảm bảo tính bảo mật.
- HTTPS bảo vệ thông tin người dùng như mật khẩu, số thẻ ngân hàng khỏi các cuộc tấn công như Man-in-the-Middle (MITM).
- HTTPS yêu cầu chứng chỉ số (SSL Certificate) từ các tổ chức xác thực danh tính (CA -Certificate Authority).

Do tính bảo mật cao, HTTPS trở thành tiêu chuẩn bắt buộc cho các trang web ngân hàng, thương mại điện tử và các dịch vụ đăng nhập trực tuyến.

CHƯƠNG 4: ỨNG DỤNG

4.1. Phương pháp kiểm thử

Trong bài báo cáo này, chúng em sẽ tiến hành xây dựng và kiểm thử trên môi trường Docker. Đây là một giải pháp ảo hóa phù hợp bởi khả năng tối ưu hóa việc sử dụng bộ nhớ. Thay vì chạy một máy ảo hoàn chỉnh, các container chỉ chứa những tính năng cần thiết cho việc kiểm thử, giúp quản lý và sử dụng bộ nhớ hiệu quả hơn.

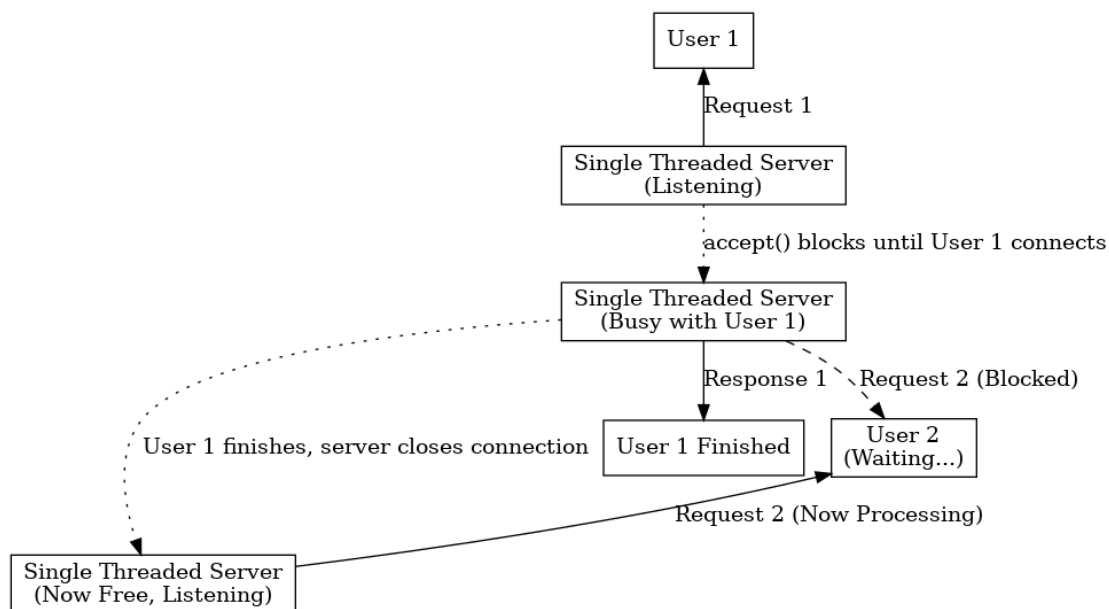
Docker được lựa chọn vì mang lại nhiều ưu điểm vượt trội so với các giải pháp ảo hóa truyền thống như máy ảo. Docker sử dụng kiến trúc container chia sẻ kernel của hệ điều hành máy chủ, giúp giảm đáng kể tài nguyên hệ thống so với máy ảo vốn yêu cầu một bản sao đầy đủ của hệ điều hành, bộ nhớ riêng, dung lượng lưu trữ lớn và thời gian khởi động kéo dài hàng phút. Ngược lại, container Docker chỉ bao gồm các thư viện cần thiết, tiêu tốn ít bộ nhớ và có thể khởi động trong vài giây. Bên cạnh đó, việc thiết lập và triển khai ứng dụng trên nhiều máy chủ thông thường là một quá trình phức tạp và dễ xảy ra lỗi do sự không đồng nhất giữa các môi trường. Docker giúp chuẩn hóa môi trường triển khai, đảm bảo ứng dụng hoạt động nhất quán trên mọi hệ thống, từ đó đơn giản hóa quá trình phát triển, kiểm thử và triển khai trong dự án.

4.2. Single-Threaded Server

4.2.1. Mô tả ứng dụng và hạn chế

Single-Threaded Web Server là loại máy chủ web cơ bản nhất, hoạt động trên một luồng (single thread) duy nhất. Toàn bộ quá trình lắng nghe kết nối mới, chấp nhận kết nối, đọc yêu cầu, xử lý và gửi phản hồi cho một client đều diễn ra tuần tự trên luồng này.

1. Luồng chính của Server thực hiện các bước sau:
2. Khởi tạo và lắng nghe kết nối trên một cổng mạng cụ thể.
3. Chờ đợi (blocking) cho đến khi có Client kết nối tới (accept()).
4. Khi có client kết nối, Server sẽ tiếp nhận, đọc toàn bộ yêu cầu từ Client đó (recv() hoặc read()).
5. Xử lý yêu cầu (ví dụ: đọc file được yêu cầu, tạo phản hồi HTTP).
6. Gửi phản hồi lại cho Client (send() hoặc write()).
7. Đóng kết nối với Client hiện tại.
8. Quay lại bước 2 để chờ client tiếp theo.



Hình 6: Sơ đồ mô tả hoạt động của Single-Threaded Server

Hạn chế chính: Mô hình này cực kỳ đơn giản nhưng hoạt động rất kém hiệu quả trong môi trường thực tế có nhiều người dùng đồng thời. Vấn đề cốt lõi là tính chất blocking:

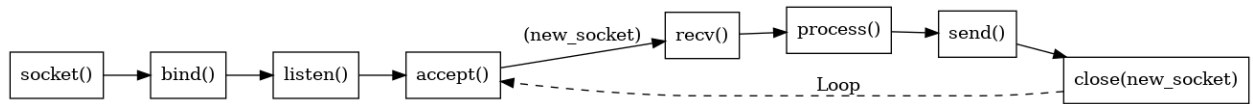
Trong suốt thời gian từ bước 3 đến bước 6 (khi Server đang xử lý một Client), nó không thể chấp nhận kết nối mới từ các client khác. Nếu client hiện tại yêu cầu một tác vụ tốn thời gian (ví dụ: đọc file lớn) hoặc có kết nối mạng chậm, tất cả các client khác đang đợi kết nối sẽ bị chặn hoàn toàn (blocked). Điều này dẫn đến trải nghiệm người dùng rất tệ (thời gian chờ đợi lâu) và hiệu năng hệ thống cực kỳ thấp khi có nhiều hơn một client truy cập.

4.2.2. Kiến trúc và các hàm cốt lõi

Kiến trúc cốt lõi của ứng dụng gồm 2 phần chính: Server và Client.

4.2.2.1 Luồng hoạt động phía Server

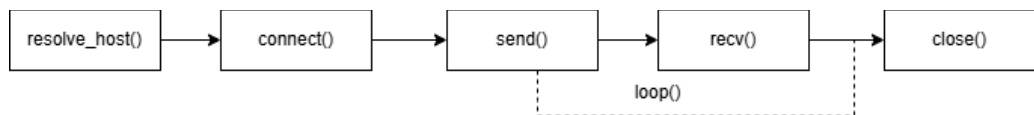
1. `socket()`: Tạo một điểm cuối giao tiếp (socket) để lắng nghe.
2. `bind()`: Gán (ràng buộc) socket vừa tạo với một địa chỉ IP và số hiệu cổng (port) cụ thể trên Server. Server lắng nghe các kết nối đến trên địa chỉ IP và cổng này.
3. `listen()`: Đánh dấu socket là sẵn sàng chấp nhận các kết nối đến và thiết lập một hàng đợi (backlog queue) cho các kết nối đang chờ được accept.
4. Vòng lặp chính (main loop):
 - a. `accept()`: Chờ (block) cho đến khi có một Client yêu cầu kết nối. Khi có kết nối, hàm này trả về một socket mới dành riêng cho việc giao tiếp với Client đó. Socket lắng nghe ban đầu vẫn giữ nguyên để tiếp tục nhận các kết nối khác, nhưng sẽ bị chờ (block) nếu Server đang bận xử lý.
 - b. `recv()/read()`: Đọc dữ liệu (được yêu cầu) từ Client thông qua socket mới đã được accept.
 - c. `process()`: Xử lý yêu cầu của Client. Ví dụ như: phân tích yêu cầu HTTP, tìm file, tạo header phản hồi.
 - d. `send()/write()`: Gửi dữ liệu (phản hồi) lại cho Client thông qua socket mới.
 - e. `close()`: Đóng socket sau khi hoàn tất giao tiếp với Client hiện tại.
 - f. Quay lại bước 4.a. để chờ Client tiếp theo.



Hình 7: Luồng hoạt động của Server trong ứng dụng Single-Threaded Server

4.2.2.2 Luồng hoạt động phía Client

1. socket(): Tạo một điểm cuối giao tiếp (socket) phía Client.
2. resolve_host(): Phân giải tên miền của Server thành địa chỉ IP (nếu Client nhập tên miền thay vì địa chỉ IP).
3. connect(): Chủ động yêu cầu kết nối tới địa chỉ IP và cổng của Server đang lắng nghe. Hàm này cũng block cho đến khi kết nối được thiết lập hoặc thất bại.
4. Vòng lặp giao tiếp:
 - a. send()/write(): Gửi yêu cầu đến Server.
 - b. recv()/read(): Nhận phản hồi từ Server.
 - c. Lặp lại nếu cần trao đổi thêm dữ liệu.
5. close(): Đóng socket phía Client khi hoàn tất.



Hình 8: Luồng hoạt động của Client trong ứng dụng Single-Threaded Server

4.2.2.3 Các hàm xử lý phía Server

```

2
3 > void error(const char *msg) { ...
6   }
7
8 > int create_socket() { ...
13  }
14
15 > void bind_socket(int sockfd, struct sockaddr_in *serv_addr) { ...
23   }
24
25 > int accept_connection(int sockfd, struct sockaddr_in *cli_addr, socklen_t
    *clilen) { ...
30   }
31
32 > void handle_client(int newsockfd) { ...
48   }
49
  
```

- a. `void error(const char *msg)`: Hàm tiện ích đơn giản để in thông báo lỗi ra `stderr` và thoát chương trình. Rất hữu ích để xử lý các lỗi từ các lời gọi hệ thống (system calls).
- b. `int create_socket()`: dùng để tạo 1 socket.
 - `socket(AF_INET, SOCK_STREAM, 0)` để tạo một socket.
 - `AF_INET`: Sử dụng họ địa chỉ IPv4.
 - `SOCK_STREAM`: Sử dụng kiểu socket stream, đảm bảo giao tiếp tin cậy, có thứ tự, hướng kết nối (phù hợp với TCP).
 - `0`: Để hệ thống tự chọn protocol mặc định cho `AF_INET` và `SOCK_STREAM`, chính là `IPPROTO_TCP`.
 - Trả về một số nguyên dương (file descriptor) đại diện cho socket nếu thành công, hoặc -1 nếu lỗi.
- c. `void bind_socket(int sockfd, struct sockaddr_in *serv_addr)`
 - `bind(sockfd, (struct sockaddr *) serv_addr, sizeof(*serv_addr))`: Hàm này liên kết socket với địa chỉ và cổng cụ thể.
 1. `sockfd`: Socket descriptor trả về từ `create_socket()`.
 2. `serv_addr`: Con trỏ tới cấu trúc `sockaddr_in` chứa thông tin địa chỉ IP và cổng của Server.
 3. `serv_addr->sin_family = AF_INET`;
 4. `serv_addr->sin_addr.s_addr = INADDR_ANY`; (Lắng nghe trên tất cả các giao diện mạng của máy chủ)
 5. `serv_addr->sin_port = htons(portno)`; (Chuyển đổi số hiệu cổng `portno` sang định dạng network byte order).
 - `listen(sockfd, 5)`;
 1. `sockfd`: Socket đã được bind.
 2. `backlog`: Số lượng kết nối tối đa có thể chờ trong hàng đợi trước khi bị từ chối (thường là 5 hoặc 10).
- d. `int accept_connection(int sockfd, struct sockaddr_in *cli_addr, socklen_t *clilen)`

- `accept(sockfd, (struct sockaddr *) cli_addr, clilen)`: Hàm này block cho đến khi có client kết nối.
 1. `sockfd`: Socket lắng nghe (đã bind và listen).
 2. `cli_addr`: Con trỏ tới cấu trúc `sockaddr_in` để lưu thông tin địa chỉ của client kết nối đến.
 3. `clilen`: Con trỏ tới biến `socklen_t` chứa kích thước của `cli_addr`.
 4. Trả về một socket descriptor mới (`newsockfd`) để giao tiếp riêng với client vừa kết nối. Socket `sockfd` ban đầu vẫn mở để tiếp tục lắng nghe. Trả về -1 nếu lỗi.

e. `void handle_client(int newsockfd)`

- Hàm này chứa logic xử lý giao tiếp với một client cụ thể thông qua `newsockfd`.
- Bên trong thường có vòng lặp để đọc (`read(newsockfd, buffer, size)`) và ghi (`write(newsockfd, response, strlen(response))`).
- Phân tích buffer nhận được để hiểu yêu cầu của client.
- Tạo response phù hợp.
- Cuối cùng, gọi `close(newsockfd)` để đóng kết nối với client này.

4.2.2.4 Các hàm xử lý Client

```

4
3 > void error(const char *msg) { ...
6   }
7
8 > int create_socket() { ...
13  }
14
15 > struct hostent *resolve_host(const char *hostname) { ...
22  }
23
24 > void connect_to_server(int sockfd, struct sockaddr_in *serv_addr, struct
    hostent *server, int portno) { ...
32  }
33
34 > void communicate_with_server(int sockfd) { ...
64  }

```

a. `create_socket()`: Tạo socket phía Client.

- `socket(AF_INET, SOCK_STREAM, 0)`: Đây là hàm để khởi tạo TCP socket cho kết nối giữa Client và Server.

- AF_INET là tham số báo cho socket biết là giao thức IP sẽ dùng họ địa chỉ là IPv4.
- SOCK_STREAM là kiểu socket. Kiểu này liên quan đến TCP, vì SOCK_STREAM cung cấp kết nối tin cậy, hướng kết nối, dòng byte.
- Tham số thứ ba là protocol, ở đây là 0. Khi đặt 0, hệ thống tự chọn protocol phù hợp dựa vào hai tham số trước. Vì AF_INET và SOCK_STREAM nên protocol mặc định là IPPROTO_TCP.
- Thành công sẽ trả về 1 số nguyên dương đại diện cho socket đó (socket file descriptor).

b. resolve_host(): Phân giải tên miền thành địa chỉ IP.

- gethostbyname(hostname): hàm sẽ trả về IPv4 (không hỗ trợ IPv6) của tên miền. Tên miền truyền vào vẫn có thể là IP của máy.
- Trả về 1 struct có cấu trúc như sau:
 - struct hostent {
 - char *h_name; // Tên chính thức của host
 - char **h_aliases; // Danh sách các tên alias (biệt hiệu)
 - int h_addrtype; // Loại địa chỉ (AF_INET cho IPv4)
 - int h_length; // Độ dài địa chỉ (4 byte cho IPv4)
 - char **h_addr_list; // Mảng các địa chỉ IP (dạng binary)
 - char *h_addr; // Địa chỉ host đầu tiên h_addr_list[0]

c. connect_to_server(): Thiết lập kết nối tới Server.

- Các struct đối số: dùng khởi tạo thông tin của socket, trong hàm thì sẽ là khởi tạo thông tin của Server để tiến hành kết nối.
 - struct sockaddr_in {
 - short sin_family; // AF_INET (IPv4)
 - unsigned short sin_port; // Port (network byte order)
 - struct in_addr sin_addr; // Địa chỉ IP
 - char sin_zero[8]; // Padding

};

○ Khởi tạo thông số cho Server:

1. `serv_addr.sin_family = AF_INET;` // họ địa chỉ IPv4
2. `bcopy((char *)server, &serv_addr->sin_addr.s_addr, server->h_length);` thực hiện sao chép địa chỉ IP của Server từ cấu trúc `hostent` (kết quả từ `gethostbyname()`) vào cấu trúc `sockaddr_in` để thiết lập kết nối.
3. `serv_addr->sin_port = htons(portno);` gán port của Server.

○ `connect(sockfd, (struct sockaddr *) serv_addr, sizeof(*serv_addr));` dùng để kết nối với Server.

d. `communicate_with_server()`: Hàm này nhận vào một socket file descriptor, được dùng để giao tiếp giữa Client và Server. Trong hàm có một vòng lặp vô hạn `while(1)`, nghĩa là client sẽ liên tục gửi và nhận dữ liệu cho đến khi client nhập "exit".

- `write(sockfd, buffer, strlen(buffer));` hàm dùng để gửi dữ liệu thông qua socket, tham số `sockfd` là socket descriptor, `buffer` là dữ liệu cần gửi, và `strlen(buffer)` là độ dài dữ liệu.
- `read(sockfd, buffer, BUFFER_SIZE - 1);` hàm dùng để nhận dữ liệu thông qua socket, tham số đầu tiên cũng là socket descriptor, `buffer` để lưu dữ liệu nhận được, và `BUFFER_SIZE - 1` là số byte tối đa đọc vào.

e. `error()`: Hàm tiện ích xử lý lỗi.

4.2.3. Các bước xây dựng ứng dụng

Đầu tiên, tiến hành cài đặt các công cụ cần thiết sau đây: gcc, make, net-tools.

```
CLIENT: docker-compose.yml | DOCKERFILE: makefile | Server
cocheche@ubuntu:~/Desktop/SP_TCP-IP/tcp_server_v1$ make

Command 'make' not found, but can be installed with:

sudo apt install make
sudo apt install make-guile

cocheche@ubuntu:~/Desktop/SP_TCP-IP/tcp_server_v1$ sudo apt update && sudo apt install make
```


Sau đó, chúng em có chuẩn bị một file Makefile dùng để biên dịch chương trình một cách nhanh và dễ dàng hơn, không cần thủ công đánh từng lệnh. Trước tiên thì xài lệnh make clean để xóa các bản build cũ, sau đó dùng make để build lại chương trình.

```
cocheche@ubuntu:~/Desktop/SP_TCP-IP/tcp_server_v1$ make clean
rm -f server/server_v1 client/client_v1 server/tcp_server.o server/socket_utils.o client/tcp_client.o client/socket_utils.o
cocheche@ubuntu:~/Desktop/SP_TCP-IP/tcp_server_v1$ make
gcc -Wall -Wextra -Werror -c server/tcp_server.c -o server/tcp_server.o
gcc -Wall -Wextra -Werror -c server/socket_utils.c -o server/socket_utils.o
gcc -Wall -Wextra -Werror -o server/server_v1 server/tcp_server.o server/socket_utils.o
gcc -Wall -Wextra -Werror -c client/tcp_client.c -o client/tcp_client.o
gcc -Wall -Wextra -Werror -c client/socket_utils.c -o client/socket_utils.o
gcc -Wall -Wextra -Werror -o client/client_v1 client/tcp_client.o client/socket_utils.o
cocheche@ubuntu:~/Desktop/SP_TCP-IP/tcp_server_v1$ cd client
```

Do môi trường sử dụng kiểm thử chương trình là Docker, nên chúng em tiến hành tạo các Container chuẩn bị sẵn trong file docker-compose.yaml bằng lệnh sau (nếu dùng 2 máy ảo thì không cần tới bước này).

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
For more help on how to use Docker, head to https://docs.docker.com/go/guides/
cocheche@ubuntu:~/Desktop/SP_TCP-IP/tcp_server_v1$ sudo docker compose -f docker-compose.yaml up -d
[+] Building 0.0s (0/0)
[+] Running 2/2
  Container tcp_server_v1-server-1 Started
    1.8s
  Container tcp_server_v1-client-1 Started
    2.8s
cocheche@ubuntu:~/Desktop/SP_TCP-IP/tcp_server_v1$
```

Tiếp theo, dùng docker exec để vào running container thực hiện chạy chương trình và thử nghiệm.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
cocheche@ubuntu:~/Desktop/SP_TCP-IP$ cd tcp_server_v1/
cocheche@ubuntu:~/Desktop/SP_TCP-IP/tcp_server_v1$ sudo docker ps
[sudo] password for cocheche:
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAME
5343784266ee   gcc-core  "/bin/sh"               31 seconds ago Up 28 seconds                               tcp_
server_v1-client-1
989bf429b071   gcc-core  "/bin/sh"               31 seconds ago Up 29 seconds  0.0.0.0:8080->8080/tcp, :::8080->8080/tcp  tcp_
server_v1-server-1
cocheche@ubuntu:~/Desktop/SP_TCP-IP/tcp_server_v1$ sudo docker exec -it 5343 /bin/sh
/app # ls
client.h          client_v1         socket_utils.c    socket_utils.o    tcp_client.c      tcp_client.o
/app #
```

Khởi động container Server.

```
2.8s
cocheche@ubuntu:~/Desktop/SP_TCP-IP/tcp_server_v1$ sudo docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
5343784266ee   gcc-core   "/bin/sh"               3 minutes ago Up 3 minutes
rver_v1-client-1  gcc-core   "/bin/sh"               3 minutes ago Up 3 minutes   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp
rver_v1-server-1

cocheche@ubuntu:~/Desktop/SP_TCP-IP/tcp_server_v1$ sudo docker exec -it 989b /bin/sh
/app # ls
server.h          server_v1          socket_utils.c     socket_utils.o     tcp_server.c       tcp_server.o
/app #
/app #
/app #
```

Trong quá trình chạy chương trình, chúng ta cần phải xác định địa chỉ IP của Server. Bước này dùng cho việc khi chạy Client sẽ cần 1 trường hostname và trường này chính là IP của Server.

```
/app #
/app #
/app # sudo apk add net-tools
/bin/sh: sudo: not found
/app # apk add net-tools
fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/main/x86_64/APKINDEX.tar.gz
fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/community/x86_64/APKINDEX.tar.gz
(1/2) Installing mii-tool (2.10-r3)
(2/2) Installing net-tools (2.10-r3)
Executing busybox-1.37.0-r12.trigger
OK: 454 MiB in 44 packages
/app # ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.20.0.100 netmask 255.255.255.0 broadcast 172.20.0.255
    ether 02:42:ac:14:00:64 txqueuelen 0 (Ethernet)
    RX packets 290 bytes 2649666 (2.5 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 223 bytes 13280 (12.9 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Sau đó chạy server_v1 và không thể chạy được do có thể container thiếu thư viện.

```
Try 'ldd --help' for more information.
cocheche@ubuntu:~/Desktop/SP_TCP-IP$ ldd tcp_server_v1/server/s
server.h          server_v1          socket_utils.c     socket_utils.o
cocheche@ubuntu:~/Desktop/SP_TCP-IP$ ldd tcp_server_v1/server/server_v1
linux-vdso.so.1 (0x00007fff607e8000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f7cdca24000)
/lib64/ld-linux-x86-64.so.2 (0x00007f7cdd018000)
cocheche@ubuntu:~/Desktop/SP_TCP-IP$
```

Cài thêm thư viện cho Server *apk add --no-cache libc6-compat*.

```
/app # apk add --no-cache libc6-compat
fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/main/x86_64/APKINDEX.tar.gz
fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/community/x86_64/APKINDEX.tar.gz
(1/3) Installing musl-obstack (1.2.3-r2)
(2/3) Installing libucontext (1.3.2-r0)
(3/3) Installing gcompat (1.1.0-r4)
OK: 454 MiB in 47 packages
/app #
```

Chạy lại chương trình.

```
(3/3) Installing gcompat (1.1.0-r4)
OK: 454 MiB in 47 packages
/app # ./server_v1
TCP server listening on port 8081
```

Tại Client chạy `client_v1 <hostname> <port>` kết nối với Server, không chạy được do thiếu thư viện như lỗi trên vì build từ ngoài máy Ubuntu chứ không phải trong container dẫn đến container có thể không chạy được do thiếu các thư viện cần thiết.

```
> cocheche@ubuntu:~/Desktop/SP_TCP-IP/tcp_server_v1$ sudo docker exec -it 5343 /bin/sh
/app # ls
client.h      client_v1    socket_utils.c  socket_utils.o  tcp_client.c    tcp_client.o
/app # ./client_v1 172.20.0.100 8081
/bin/sh: ./client_v1: not found
/app #
```

Cài thư viện `apk add --no-cache libc6-compat` tương tự như Server.

```
/bin/sh: ./client_v1: not found
/app # apk add --no-cache libc6-compat
fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/main/x86_64/APKINDEX.tar.gz
fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/community/x86_64/APKINDEX.tar.gz
(1/3) Installing musl-obstack (1.2.3-r2)
(2/3) Installing libucontext (1.3.2-r0)
(3/3) Installing gcompat (1.1.0-r4)
OK: 454 MiB in 45 packages
/app #
```

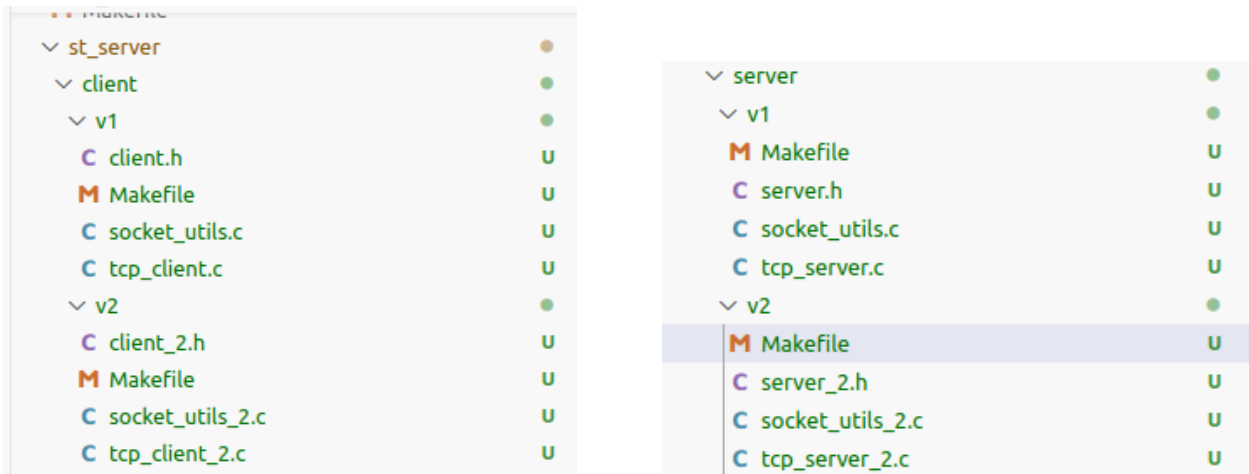
Thử nghiệm Client gửi một message đến máy chủ Server như sau.

```
(3/3) Installing gcompat (1.1.0-r4)
OK: 454 MiB in 45 packages
/app # ./client_v1 172.20.0.100 8081
Please enter the message: hello
Server response: Message received
Please enter the message: 
```

Khi Client hoàn tất việc gửi, Server đã lắng nghe được, gửi phản hồi về Client và ngắt kết nối.

```
OK: 454 MiB in 47 packages
/app # ./server_v1
TCP server listening on port 8081
Message from client: hello
----
```

Ta có thể thấy việc này có phần bất hợp lý và không giống một Server thực thụ, nên ta tiến hành cập nhật Server liên tục lắng nghe các kết nối từ Client. Để giải quyết vấn đề này thì chúng em có ý tưởng sẽ tạo vòng lặp `while` để có thể liên tục chấp nhận kết nối từ Client cho đến khi Client ra dấu hiện báo thoát kết nối, chỉnh sửa lại cây thư mục project như sau.

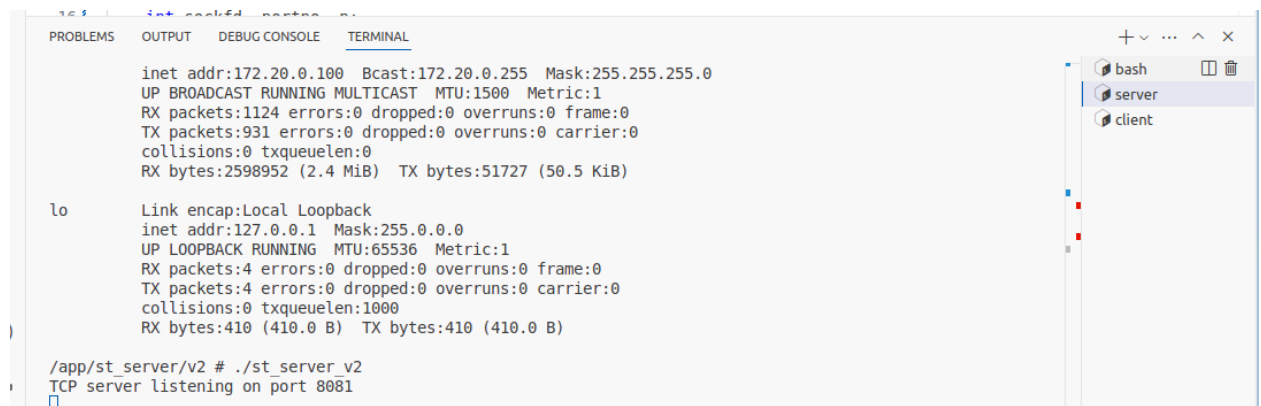


Để cải thiện việc lắng nghe của Single-threaded Server thì chúng em, tiến hành tạo 1 vòng lặp vô tận đến khi có dấu hiệu dừng lại thì nó sẽ kết thúc. Tại chương trình này thì Server sẽ luôn tục lắng nghe kết nối tới client và cũng sẽ có thể phản hồi lại client.

```
// Vòng lặp giao tiếp với client
while (1) {
    if (receive_message(newsockfd, buffer, sizeof(buffer)) == 0) {
        printf("Client disconnected\n");
        break;
    }
    printf("Message from client: %s\n", buffer);

    send_message(newsockfd, "Message received");
}
```

Để xây dựng chương trình thì ta cũng tiến hành chạy Makefile đã được chuẩn bị sẵn để biên dịch. Ban đầu ta tiến hành khởi chạy Server.



Sau đó chạy Client kết nối tới Server này và gửi một số message như ảnh.

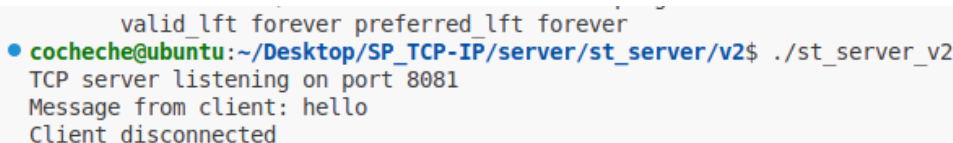


```
OK: 454 MiB in 45 packages
/app/st_client/v2 # cd v2
/bin/sh: cd: can't cd to v2: No such file or directory
/app/st_client/v2 # ls
client_2.h      socket_utils_2.c  socket_utils_2.o  st_client_v2      tcp_client_2.c    tcp_client_2.o
/app/st_client/v2 # ./st_client_v2 172.20.0.100 8081
hello
Please enter the message (or type 'exit' to quit): Server response: Message received
hello
Please enter the message (or type 'exit' to quit): Server response: Message received
exit
Please enter the message (or type 'exit' to quit): Exiting...
/app/st_client/v2 # ./st_client_v2 172.20.0.100 8081
Hello from client
Please enter the message (or type 'exit' to quit): Server response: Message received
Hi nice to meet you!
Please enter the message (or type 'exit' to quit): Server response: Message received
```

Tại Server, chúng ta có thể nhận các tin nhắn từ Client và phản hồi lại bằng một thông báo đã được chuẩn bị sẵn, xác nhận rằng tin nhắn đã được tiếp nhận.



```
cocheche@ubuntu:~/Desktop/SP_TCP-IP$ sudo docker start 8aaa
8aaa
cocheche@ubuntu:~/Desktop/SP_TCP-IP$ sudo docker exec -it 8aaa /bin/sh
/app # ls
http_server  https_server  mt_server     st_server
/app # cd st_server/
/bin/sh: cd: not found
/app # cd st_server/
/app/st_server # cs v2
/bin/sh: cs: not found
/app/st_server # cd v2
/app/st_server/v2 # ls
server_2.h      socket_utils_2.c  socket_utils_2.o  st_server_v2      tcp_server_2.c    tcp_server_2.o
/app/st_server/v2 # ./st_server_v2
TCP server listening on port 8081
Message from client: Hello from client
Message from client: Hi nice to meet you!
```



```
valid_lft forever preferred_lft forever
cocheche@ubuntu:~/Desktop/SP_TCP-IP/server/st_server/v2$ ./st_server_v2
TCP server listening on port 8081
Message from client: hello
Client disconnected
```

4.2.4. Kết luận

Việc triển khai và kiểm nghiệm thành công mô hình Server đơn luồng cơ bản bằng ngôn ngữ C trong môi trường Docker đánh dấu một bước tiến quan trọng trong đồ án. Phần này đã mang lại những hiểu biết thực tiễn cần thiết về các hàm cốt lõi của Socket API (socket, bind, listen, accept, read/write, close) cũng như quy trình xử lý tuần tự kết nối theo mô hình TCP.

Tuy nhiên, quá trình thử nghiệm cũng làm nổi bật những hạn chế nghiêm trọng của kiến trúc đơn luồng, đặc biệt là tính blocking của các lời gọi I/O như accept và read. Điều này dẫn đến việc Server không thể tiếp nhận hoặc xử lý các kết nối mới trong khi đang phục vụ một client hiện tại, gây ra hiệu suất thấp và khả năng đáp ứng đồng thời kém. Do đó, kiến trúc này không thích hợp cho các ứng dụng thực tế cần xử lý nhiều client cùng lúc.

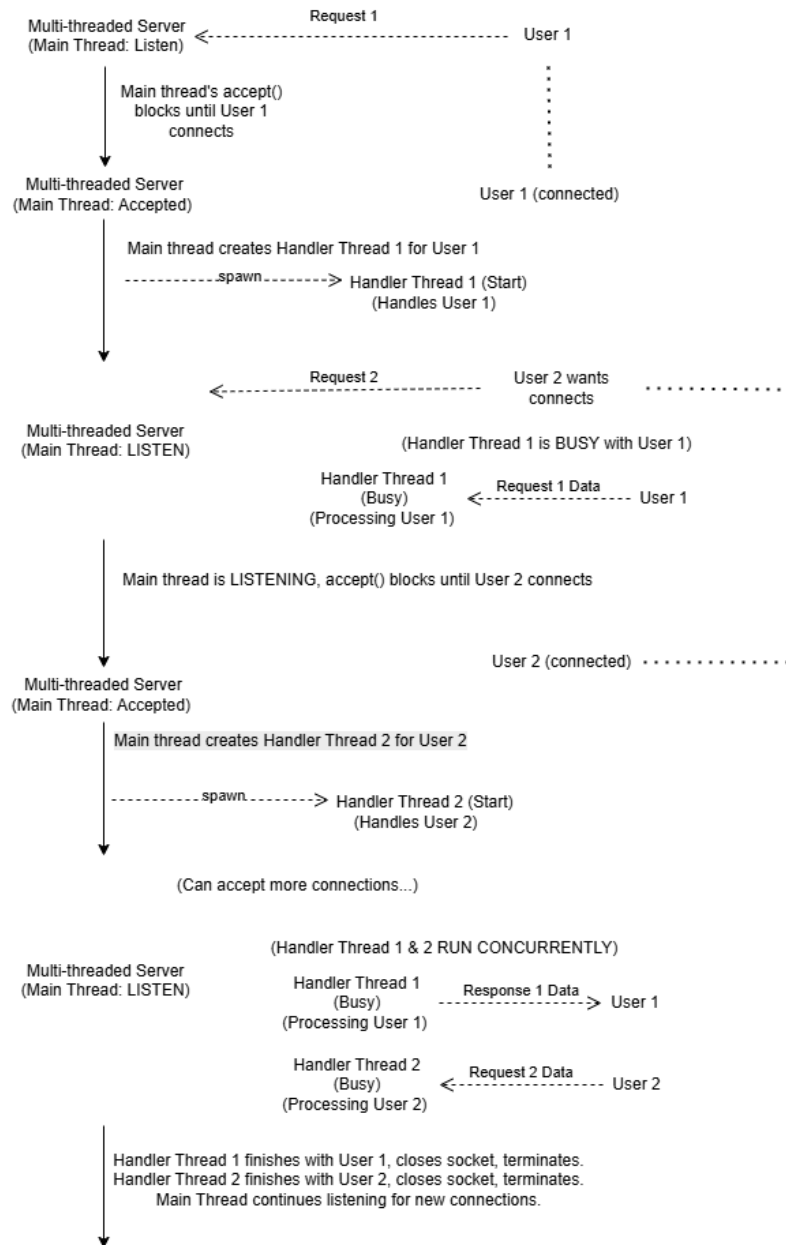
Những hạn chế này đã chỉ ra sự cần thiết phải cải tiến chương trình. Do đó, phần tiếp theo của đồ án sẽ tập trung vào việc phát triển và triển khai mô hình Server đa luồng, nhằm khắc phục vấn đề blocking, cho phép xử lý song song các yêu cầu từ nhiều Client, từ đó cải thiện đáng kể hiệu suất và khả năng mở rộng của hệ thống. Nền tảng kiến thức và kinh nghiệm thu được từ việc xây dựng Server đơn luồng này sẽ là cơ sở quan trọng cho giai đoạn phát triển tiếp theo.

4.3. Multi-Threaded Server

4.3.1. Mô tả ứng dụng và hạn chế

Multi-Threaded Server là một phiên bản nâng cấp của Server trước đó, giải quyết được vấn đề blocking bằng cách sử dụng đa luồng để xử lý đồng thời nhiều kết nối từ Client. Server này đã phần nào giải quyết được vấn đề cốt lõi của Server trước. Trong khi Client 1 kết nối và giao tiếp đến Server, Server vẫn tiếp tục lắng nghe và mở kết nối mới nếu có một Client 2 kết nối tới nó.

Cụ thể, khi một Client mới kết nối đến Server, luồng chính (main thread) sẽ thực hiện việc chấp nhận kết nối (thông qua hàm accept()). Sau đó, luồng chính sẽ ngay lập tức tạo ra một luồng riêng biệt (thread) để xử lý giao tiếp với Client đó. Điều này cho phép luồng chính tiếp tục lắng nghe và chấp nhận các kết nối mới mà không cần phải chờ luồng xử lý giao tiếp với Client hoàn tất. Mỗi luồng phụ trách đọc yêu cầu từ Client (sử dụng hàm read()), xử lý thông tin (trong trường hợp này là chuẩn bị dữ liệu echo) và gửi phản hồi (bằng hàm write()) cho Client tương ứng. Cuối cùng, nó sẽ đóng kết nối (bằng hàm close()) sau khi hoàn thành. Phương pháp này cho phép Server phục vụ nhiều client đồng thời, từ đó cải thiện đáng kể khả năng đáp ứng và hiệu suất so với mô hình đơn luồng.



Hình 9: Sơ đồ mô tả hoạt động của Multi-Threaded Server

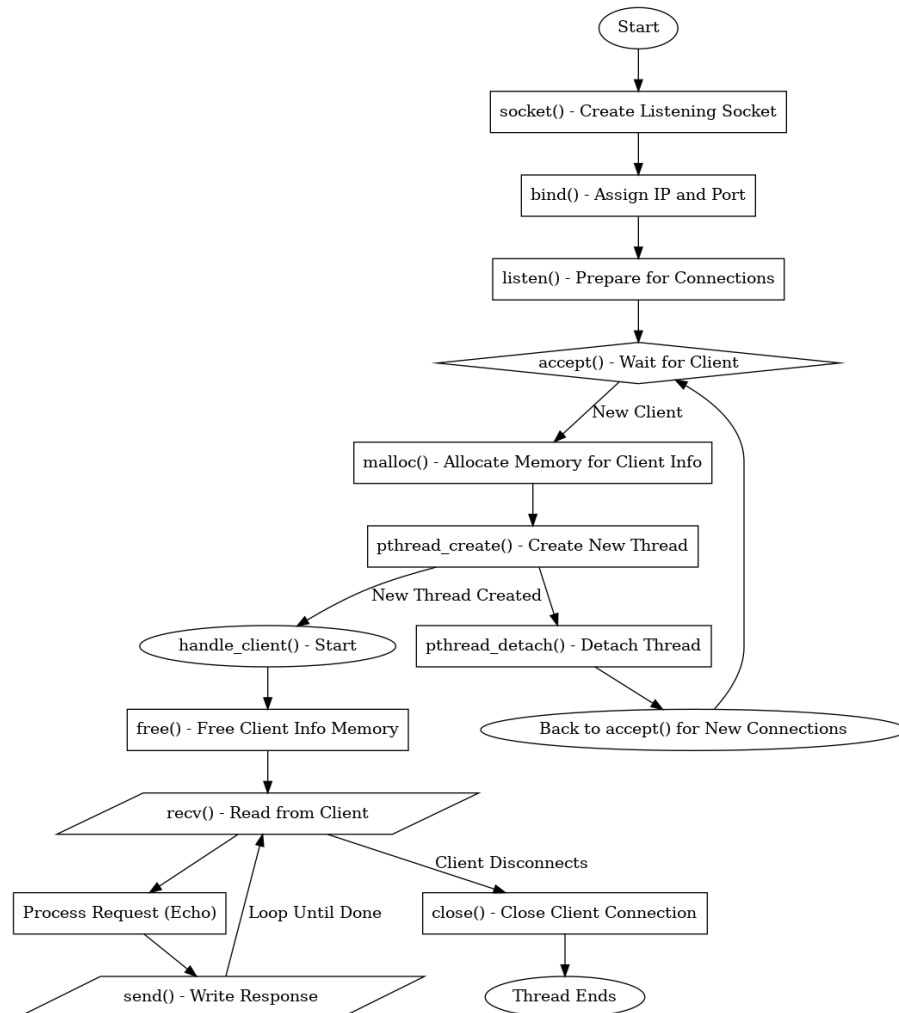
Hạn chế chính của ứng dụng: Mặc dù server đa luồng xử lý được nhiều người dùng cùng lúc, nó vẫn có một số điểm yếu:

- **Tốn tài nguyên:** Mỗi người dùng kết nối cần một luồng riêng. Nếu có quá nhiều người (hàng nghìn trở lên), việc tạo và chạy tất cả các luồng đó sẽ tốn rất nhiều bộ nhớ và làm chậm máy chủ do phải liên tục chuyển đổi qua lại giữa các luồng.

- Khó quản lý: Việc theo dõi và điều khiển hàng nghìn luồng chạy cùng lúc phức tạp hơn nhiều so với chỉ một luồng.
- Tạo/Hủy luồng liên tục tốn kém: Server cứ tạo luồng mới khi có người vào và hủy đi khi họ ra. Nếu người dùng vào/ra liên tục, việc này gây lãng phí tài nguyên. Tốt hơn là nên có một "hồ bơi luồng" (thread pool) - các luồng được tạo sẵn để dùng lại.

Đễ bị lỗi nếu các luồng dùng chung dữ liệu: Khi nhiều luồng cùng truy cập hoặc thay đổi một dữ liệu dùng chung, chúng có thể gây ra lỗi khó lường (gọi là "race condition"). Để tránh lỗi này, lập trình viên phải dùng các kỹ thuật khóa (như mutex) cẩn thận, làm code phức tạp hơn.

4.3.2. Kiến trúc và các hàm cốt lõi



4.3.2.1 *Luồng hoạt động phía Server*

1. Luồng Chính (Main Thread):

- a. `socket()`: Tạo socket lắng nghe.
- b. `bind()`: Gán socket với địa chỉ IP và cổng.
- c. `listen()`: Đánh dấu socket sẵn sàng nhận kết nối, tạo hàng đợi.
- d. Vòng lặp chính (Main Loop):
 - `accept()`: Chờ (block) và chấp nhận kết nối mới từ client. Trả về một socket mới (`new_socket`) cho client đó.
 - `malloc()`: Quan trọng: Cấp phát bộ nhớ động cho cấu trúc `client_info_t` để lưu trữ thông tin của client vừa kết nối (gồm `new_socket` và `client_addr`).
 - `pthread_create()`: Điểm khác biệt chính: Tạo một luồng mới để thực thi hàm `handle_client`. Con trỏ tới `client_info_t` được cấp phát ở trên được truyền làm đối số cho luồng mới. Luồng chính không chờ luồng này hoàn thành.
 - `pthread_detach()`: Tách luồng vừa tạo ra khỏi luồng chính.

2. Luồng Xử lý Client (Client Handler Thread - thực thi hàm `handle_client`):

- a. Hàm `handle_client(void *arg)` nhận con trỏ `arg` (là con trỏ `client_info_t`).
- b. Ép kiểu `arg` về `client_info_t *` và trích xuất `client_socket`, `client_addr`.
- c. `free(arg)`: Quan trọng: Giải phóng bộ nhớ của `client_info_t` đã được cấp phát bởi luồng chính.
- d. `inet_ntop()` / `ntohs()`: Lấy thông tin IP/Port của client để hiển thị (tùy chọn).
- e. Vòng lặp giao tiếp:
 - `read()` / `recv()`: Đọc dữ liệu từ `client_socket` của client này. Lệnh này chỉ block luồng client hiện tại, không ảnh hưởng đến luồng chính hay các luồng client khác.

- `write()` / `send()`: Gửi dữ liệu (phản hồi echo) trở lại client này thông qua `client_socket`. Lệnh này cũng chỉ block luồng client hiện tại.
 - Vòng lặp kết thúc khi `read()` trả về 0 (client ngắt kết nối) hoặc < 0 (lỗi).
- f. `close()`: Đóng `client_socket` của client này khi giao tiếp kết thúc.
- g. Luồng kết thúc (trả về NULL). Tài nguyên được tự động thu hồi do đã detach.

4.3.2.2 *Luồng hoạt động phía Client*

Không thay đổi so với client phiên bản trước vẫn thực hiện `socket()`, `connect()`, và vòng lặp `send()/recv()`, `close()`.

4.3.3. **Các bước xây dựng ứng dụng**

Đầu tiên là chuẩn bị môi trường và công cụ: Đảm bảo đã cài đặt gcc, make, và docker, docker-compose (nếu sử dụng Docker).

```

bt-dns-client-1
• cocheche@ubuntu:~/Desktop/SP_TCP-IP$ sudo apt install gcc make
Reading package lists... Done
Building dependency tree
Reading state information... Done
make is already the newest version (4.1-9.1ubuntu1).
gcc is already the newest version (4:7.4.0-1ubuntu2.3).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
○ cocheche@ubuntu:~/Desktop/SP_TCP-IP$

```

Sau đó, biên dịch mã nguồn dựa trên Makefile đã chuẩn bị sẵn. Dùng lệnh *make clean* để xóa các tệp biên dịch trước đó rồi sử dụng lệnh *make* để biên dịch lại.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
cocheche@ubuntu:~/Desktop/SP_TCP-IP$ make clean
rm -f client/st_client/v1/tcp_client.o client/st_client/v1/socket_utils.o client/st_client/v2/tcp_client_2.o
client/st_client/v2/socket_utils_2.o server/st_server/v1/tcp_server.o server/st_server/v1/socket_utils.o serv
er/st_server/v2/tcp_server_2.o server/st_server/v2/socket_utils_2.o client/st_client/v1/st_client_v1_client/s
t_client/v2/st_client_v2 server/st_server/v1/st_server_v1 server/st_server/v2/st_server_v2 client/mt_client/c
lient_utils.o client/mt_client/client.o server/mt_server/server_utils.o server/mt_server/server.o server/mt_s
erver/client_handler.o client/mt_client/mt_client server/mt_server/mt_server
cocheche@ubuntu:~/Desktop/SP_TCP-IP$ make
gcc -Wall -Wextra -O2 -c client/st_client/v1/tcp_client.c -o client/st_client/v1/tcp_client.o
gcc -Wall -Wextra -O2 -c client/st_client/v1/socket_utils.c -o client/st_client/v1/socket_utils.o
client/st_client/v1/socket_utils.c: In function 'communicate_with_server':
client/st_client/v1/socket_utils.c:43:9: warning: ignoring return value of 'fgets', declared with attribute w
arn_unused_result [-Wunused-result]
    fgets(buffer, BUFFER_SIZE - 1, stdin);
    ^
gcc -Wall -Wextra -O2 -o client/st_client/v1/st_client_v1 client/st_client/v1/tcp_client.o client/st_client/v
1/socket_utils.o
```

Chạy file Docker khởi tạo các container và network được chuẩn bị sẵn bằng *docker*
compose -f docker-compose.yaml up -d

```
cocheche@ubuntu:~/Desktop/SP_TCP-IP$ sudo docker compose -f docker-compose.yaml up -d
[+] Building 0.0s (0/0)
[+] Running 2/2
✓ Container sp_tcp-ip-server-1 Started 2.4s
✓ Container sp_tcp-ip-client-1 Started 1.4s
cocheche@ubuntu:~/Desktop/SP_TCP-IP$ sudo docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
NAMES
6b202c42bfbe   gcc-core   "/bin/sh"               13 seconds ago Up 11 seconds
sp_tcp-ip-client-1
86fc59d4b565   gcc-core   "/bin/sh"               13 seconds ago Up 10 seconds   0.0.0.0:8080->8080/tcp, :::8080->8080/
tcp
sp_tcp-ip-server-1
cocheche@ubuntu:~/Desktop/SP_TCP-IP$
```

Kiểm tra các container đang chạy và tiến hành exec vào container Server và Client.
Sau đó cài đặt những thư viện còn thiếu như là *apk add --no-cache libc6-compat net-tools*.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
0->8080/tcp, :::8080->8080/tcp sp_tcp-ip-server-1
cocheche@ubuntu:~/Desktop/SP_TCP-IP$ sudo docker exec -it 86fc /bin/sh
/app # ls
http_server  https_server  mt_server     st_server
/app #
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
/app # apk add --no-cache libc6-compat net-tools
fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/main/x86_64/APKINDEX.tar.gz
fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/community/x86_64/APKINDEX.tar.gz
(1/5) Installing musl-obstack (1.2.3-r2)
(2/5) Installing libucontext (1.3.2-r0)
(3/5) Installing gcompat (1.1.0-r4)
(4/5) Installing mii-tool (2.10-r3)
(5/5) Installing net-tools (2.10-r3)
Executing busybox-1.37.0-r12.trigger
OK: 454 MiB in 47 packages
/app #
```

Tại Server, xác định IP của Server và chạy file Server để lắng nghe các kết nối từ phía Client. Khi chạy Server sẽ hiển thị thông báo “Server listening on port 8080...”.

```
/app # cd mt_server/  
/app/mt_server # ls  
client_handler.c  include          server.c          server_utils.c  
client_handler.o  mt_server       server.o          server_utils.o  
/app/mt_server # ./mt_server  
Server listening on port 8080...
```

Tại Client, ta cũng tiến hành truy cập vào shell của Client và cài đặt các thư viện còn thiếu tương tự Server.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  
  
sudo: exec: command not found  
cocheche@ubuntu:~/Desktop/SP_TCP-IP$ sudo docker exec -it 6b20 /bin/sh  
/app # ls  
http_client  https_client  mt_client  st_client  
/app # là apk add --no-cache libc6-compat net-tools  
/bin/sh: là: not found  
/app # apk add --no-cache libc6-compat net-tools  
fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/main/x86_64/APKINDEX.tar.gz  
fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/community/x86_64/APKINDEX.tar.gz  
(1/5) Installing musl-obstack (1.2.3-r2)  
(2/5) Installing libucontext (1.3.2-r0)  
(3/5) Installing gcompat (1.1.0-r4)  
(4/5) Installing mii-tool (2.10-r3)  
(5/5) Installing net-tools (2.10-r3)  
Executing busybox-1.37.0-r12.trigger  
OK: 454 MiB in 47 packages  
/app #
```

Chạy file thực thi Client, cung cấp IP và Port của Server: ./client 172.20.0.100 8080. Nhập tin nhắn, nhấn Enter. Server (trong cửa sổ terminal kia) sẽ hiển thị tin nhắn nhận được, và Client sẽ nhận lại tin nhắn echo.

```
http_client  https_client  mt_client  st_client  
/app # cd mt_client/  
/app/mt_client # ls  
client.c      client.o      client_utils.o  
client.h      client_utils.c  mt_client  
/app/mt_client # ./mt_client 172.20.0.100 8080  
Hello from client 1  
Enter message (or type 'exit' to quit): Server response: Hello from client 1
```

Chạy Client 2 (Đồng thời): Trong khi Client 1 vẫn đang kết nối, mở một terminal khác, truy cập lại shell của container client (hoặc một container client khác nếu có) và chạy lại lệnh ./client 172.20.0.100 8080.

```
/app # cd mt_client/
/app/mt_client # ls
client.c          client.o          client_utils.o
client.h          client_utils.c   mt_client
/app/mt_client # ./mt_client 172.20.0.100 8080
Hello from client 2
Enter message (or type 'exit' to quit): Server response: Hello from client 2
```

Ta có thể gửi tin nhắn từ cả hai client. Server sẽ xử lý chúng đồng thời (output trên console Server có thể xen kẽ). Mỗi client sẽ nhận được phản hồi echo cho tin nhắn của chính nó. Điều này chứng minh khả năng xử lý đa luồng của Server.

```
/app # cd mt_server/
/app/mt_server # ls
client_handler.c  include          server.c          server_utils.c
client_handler.o  mt_server       server.o          server_utils.o
/app/mt_server # ./mt_server
Server listening on port 8080...

New connection from 172.20.0.101:42136
Received from 172.20.0.101:42136: Hello from client 1
New connection from 172.20.0.101:53316
Received from 172.20.0.101:53316: Hello from client 2
```

Nhập exit trong mỗi Client để ngắt kết nối. Server sẽ ghi nhận Client disconnected.

```
client.n          client_utils.c   mt_client
/app/mt_client # ./mt_client 172.20.0.100 8080
Hello from client 1
Enter message (or type 'exit' to quit): Server response: Hello from client 1
exit
Enter message (or type 'exit' to quit): Exiting...
/app/mt client #
```

```
/app/mt_server # ./mt_server
Server listening on port 8080...

New connection from 172.20.0.101:42136
Received from 172.20.0.101:42136: Hello from client 1
New connection from 172.20.0.101:53316
Received from 172.20.0.101:53316: Hello from client 2
Client 172.20.0.101:42136 disconnected
```

4.3.4. Kết luận

Mô hình máy chủ đa luồng đã giải quyết thành công vấn đề blocking của Server đơn luồng, cho phép xử lý đồng thời nhiều client, cải thiện đáng kể hiệu năng và khả năng đáp ứng. Tuy nhiên, mô hình này cũng bộc lộ những hạn chế mới liên quan đến việc quản lý và tiêu tốn tài nguyên khi số lượng luồng tăng cao, cũng như sự phức tạp tiềm ẩn của việc

đồng bộ hóa dữ liệu. Đây là nền tảng quan trọng cho các máy chủ phức tạp hơn nhưng cần được tối ưu hóa thêm cho các ứng dụng quy mô lớn.

4.4. HTTP Server

4.4.1. Mô tả ứng dụng

Phần này của dự án trình bày quá trình nhóm chúng em thiết kế và xây dựng một máy chủ HTTP (HTTP Server) riêng. Ý tưởng này xuất phát từ thực tế rằng việc triển khai một máy chủ web thông thường cho các tác vụ đơn giản thường tiêu tốn nhiều tài nguyên hơn mức cần thiết. Để tối ưu hóa hiệu suất và giảm thiểu chi phí tài nguyên hệ thống, chúng em hướng đến việc tạo ra một máy chủ HTTP chỉ tập trung vào các chức năng cốt lõi. Việc tự xây dựng máy chủ này không chỉ giúp chúng em tạo ra một giải pháp phù hợp hơn cho các nhu cầu cụ thể, mà còn mang lại hiểu biết sâu sắc về cách thức hoạt động của giao thức HTTP và kiến trúc client-server.

HTTP Server (Multi-threaded, Static & PHP-CGI) là một máy chủ web hoạt động trên giao thức HTTP (không mã hóa), sử dụng kiến trúc đa luồng để xử lý đồng thời nhiều kết nối từ client. Máy chủ lắng nghe trên một cổng được định nghĩa (mặc định là 8080) và có khả năng:

1. Phân tích yêu cầu HTTP đến để xác định phương thức (GET, POST...), đường dẫn (URI), phiên bản giao thức, các header và phần body (nếu có).
2. Đối với yêu cầu tệp tĩnh (ví dụ: .html, .css, .jpg) bằng phương thức GET: Xác định vị trí tệp trên hệ thống tệp dựa vào DOCUMENT_ROOT và đường dẫn yêu cầu, kiểm tra sự tồn tại và quyền đọc, xác định loại MIME, sau đó gửi phản hồi HTTP 200 OK cùng với nội dung tệp.
3. Đối với yêu cầu tệp .php: Kích hoạt cơ chế CGI bằng cách:
 - Thiết lập các biến môi trường tiêu chuẩn CGI (ví dụ: REQUEST_METHOD, SCRIPT_FILENAME, QUERY_STRING, CONTENT_LENGTH, CONTENT_TYPE) và các biến HTTP_* từ header của request.
 - Sử dụng fork() để tạo một tiến trình con.

- Tiến trình con thực thi php-cgi (execvp), với stdin được nối với body của request (nếu là POST) và stdout được nối vào một pipe.
- Tiến trình cha (luồng xử lý client ban đầu) đọc toàn bộ output từ pipe (bao gồm cả header do PHP tạo ra và nội dung HTML), phân tích header Status: (nếu có) để xác định mã trạng thái HTTP, sau đó xây dựng và gửi phản hồi HTTP cuối cùng về cho client.

4.4.2. **Luồng hoạt động**

1. Khởi tạo (start_server): Server tạo socket, cấu hình để tái sử dụng địa chỉ (SO_REUSEADDR), bind vào địa chỉ INADDR_ANY và cổng HTTP_PORT, sau đó bắt đầu lắng nghe (listen).
2. Vòng lặp chính (start_server): Liên tục chờ và chấp nhận (accept) các kết nối TCP mới từ client.
3. Tạo luồng (start_server): Với mỗi kết nối được chấp nhận, Server cấp phát bộ nhớ cho struct client_info (chỉ chứa client_fd), tạo một luồng mới (pthread_create) để chạy hàm handle_client, và tách luồng đó ra (pthread_detach). Luồng chính ngay lập tức quay lại chờ kết nối tiếp theo.
4. Xử lý Client (handle_client): Hàm này chạy trong luồng con. Nó đọc dữ liệu yêu cầu từ client_fd bằng read(). Nếu đọc thành công, nó gọi handle_request để xử lý. Cuối cùng, nó đóng client_fd và giải phóng bộ nhớ client_info.
5. Xử lý Request (handle_request):
 - Phân tích dòng đầu (sscanf) để lấy method, path, protocol.
 - Tách query string (phần sau ?) khỏi path, đặt vào biến môi trường QUERY_STRING.
 - Gọi parse_headers để lấy khối header.
 - Nếu là POST/PUT, tìm và sao chép phần body (sau \r\n\r\n).
 - Xử lý path / (ưu tiên index.php rồi đến index.html).
 - Xây dựng đường dẫn tệp đầy đủ (file_path) từ DOCUMENT_ROOT và path.

- Kiểm tra sự tồn tại (access). Nếu không có, gửi lỗi 404 (send_response).
- Nếu là file .php, gọi handle_php_request.
- Nếu là file tĩnh và method là GET: Mở file (open), lấy kích thước (fstat), lấy MIME type (get_mime_type), gửi header 200 OK (write), đọc và gửi nội dung file (read/write), đóng file (close).
- Nếu là file tĩnh nhưng method khác GET, gửi lỗi 405.
- Giải phóng bộ nhớ headers/body (nếu được cấp phát).

6. Xử lý PHP CGI (handle_php_request):

- Thiết lập các biến môi trường CGI (setenv, set_cgi_headers).
- Tạo pipe cho output (pipe).
- Tạo tiến trình con (fork).
- Con: Đóng đầu đọc pipe output, dùng dup2 để stdout ghi vào pipe. Nếu có body, tạo pipe input, ghi body vào, dup2 để stdin đọc từ pipe input. Gọi execlp("php-cgi", ...).
- Cha: Đóng đầu ghi pipe output. Đọc toàn bộ output từ pipe (read) vào buffer động. Chờ con kết thúc (waitpid). Phân tích output (tìm \r\n\r\n), tách header CGI và body. Tìm header Status:. Xây dựng và gửi phản hồi HTTP cuối cùng (write). Giải phóng buffer.

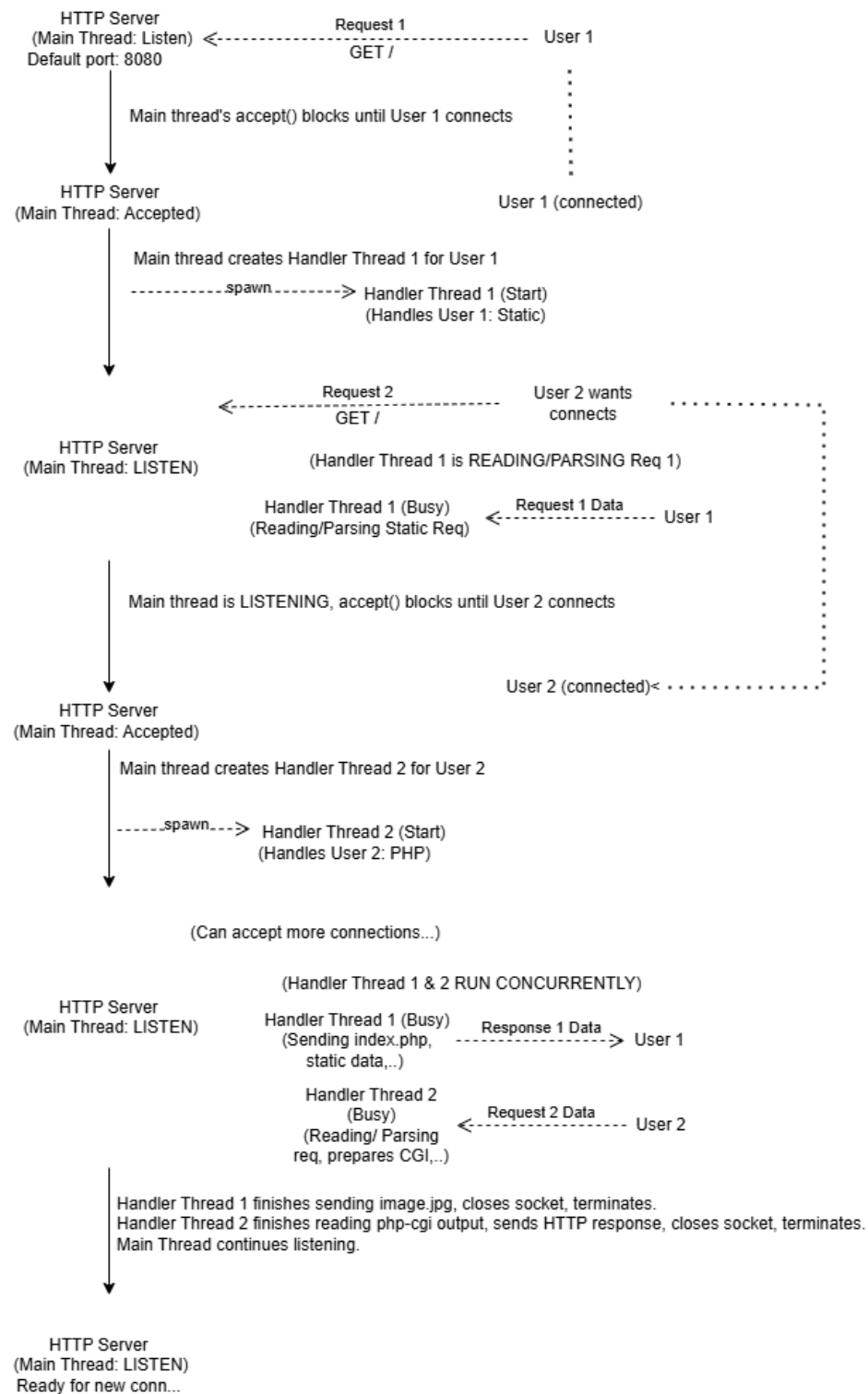
4.4.3. Hạn chế chính của ứng dụng:

- Chỉ hỗ trợ HTTP: Không có mã hóa SSL/TLS, toàn bộ giao tiếp diễn ra qua văn bản rõ, không an toàn cho dữ liệu nhạy cảm.
- Hiệu năng PHP-CGI kém: Việc tạo một tiến trình php-cgi mới cho mỗi yêu cầu PHP tốn nhiều tài nguyên hệ thống (thời gian khởi tạo, bộ nhớ) và không có khả năng mở rộng tốt như các giải pháp hiện đại (FastCGI, PHP-FPM, mod_php).
- Mô hình Thread-Per-Client: Mặc dù xử lý đồng thời, việc tạo một luồng mới cho mỗi kết nối có thể dẫn đến cạn kiệt tài nguyên (bộ nhớ, giới hạn số luồng)

khi có lượng lớn kết nối đồng thời. Các mô hình như Thread Pool hoặc Non-blocking I/O (với epoll, select) sẽ hiệu quả hơn.

- Giới hạn tính năng HTTP: Chỉ xử lý cơ bản GET/POST. Không hỗ trợ các tính năng quan trọng như Keep-Alive (duy trì kết nối cho nhiều yêu cầu), Chunked Transfer Encoding (gửi dữ liệu không biết trước độ dài), HTTP Pipelining, Compression (gzip), các cơ chế caching phức tạp.
- Bảo mật cơ bản: Việc xây dựng đường dẫn (sprintf(file_path, ..., "%s%s", DOCUMENT_ROOT, path)) có thể tiềm ẩn nguy cơ Path Traversal nếu biến path không được lọc kỹ (ví dụ chứa ../). Việc xử lý header và body cũng cần kiểm tra kỹ lưỡng hơn để tránh các tấn công như Request Smuggling hoặc tràn bộ đệm.
- Xử lý lỗi đơn giản: Chủ yếu gửi các mã lỗi HTTP cơ bản. Output lỗi từ php-cgi có thể không được chuyển tiếp đầy đủ hoặc thân thiện tới client. Không có cơ chế logging chi tiết.
- Cấu hình cứng: Các tham số như DOCUMENT_ROOT, HTTP_PORT, BUFFER_SIZE được định nghĩa trong code hoặc header, không linh hoạt.
- Giới hạn Buffer: Việc đọc request vào một buffer BUFFER_SIZE cố định có thể thất bại nếu header hoặc body của request quá lớn. Việc đọc output từ php-cgi cũng cấp phát lại bộ đệm nhưng có thể không tối ưu.

4.4.4. Kiến trúc và các hàm cốt lõi



Hình 10: Sơ đồ mô tả hoạt động của HTTP Server

1. server.c:

- a. `start_server()`: Hàm chính khởi tạo socket, lắng nghe và vòng lặp chấp nhận kết nối, tạo luồng.
- b. `handle_client(void *arg)`: Hàm thực thi trong mỗi luồng con, chịu trách nhiệm đọc request và gọi hàm xử lý `handle_request`, sau đó dọn dẹp

2. request_handler.c:

- a. `handle_request(int client_fd, const char *request)`: Hàm trung tâm, phân tích yêu cầu HTTP, xác định loại (tĩnh/PHP) và gọi hàm xử lý tương ứng hoặc gửi file tĩnh.
- b. `handle_php_request(int client_fd, const char *file_path, const char *method, const char *body, const char *headers)`: Triển khai logic CGI để thực thi script PHP thông qua php-cgi.
- c. `parse_headers(const char *request, char **headers)`: Trích xuất khối header từ yêu cầu thô.
- d. `set_cgi_headers(const char *headers)`: Chuyển đổi các HTTP header thành biến môi trường CGI HTTP_*.

3. response_handler.c:

- a. `send_response(int client_fd, const char *status, const char *content_type, const char *body)`: Gửi các phản hồi HTTP đơn giản, thường dùng cho các thông báo lỗi.

4. utils.c:

- a. `get_mime_type(const char *file_ext)`: Ánh xạ phần mở rộng của tệp sang chuỗi MIME type tương ứng.
- b. `url_decode(const char *src)`: (Có trong code nhưng không được gọi trong luồng xử lý chính của `handle_request`).

4.4.5. Các bước xây dựng ứng dụng

Chuẩn bị môi trường

- Cài đặt trình biên dịch C (gcc) và công cụ build (make).
- Cài đặt php-cgi: Đây là thành phần bắt buộc để Server có thể thực thi script PHP. Trên hệ thống Debian/Ubuntu: `sudo apt update && sudo apt install php-cgi`. Trên Alpine Linux (thường dùng trong Docker): `apk update && apk add php-cgi`.

```
rc/server.o server/https_server/src/utils.o
o cocheche@ubuntu:~/Desktop/SP_TCP-IP$ sudo apt update && sudo apt install gcc make php-cgi
```

- (Nếu dùng Docker) Chuẩn bị Dockerfile để cài đặt gcc, make, libc-dev (hoặc tương đương) và php-cgi. Chuẩn bị docker-compose.yml nếu cần quản lý nhiều container.

```
server > http_server > Dockerfile
1 FROM ubuntu:20.04
2
3 # Install dependencies
4 RUN apt-get update && apt-get install -y \
5     gcc make php-cgi spawn-fcgi nginx \
6     && rm -rf /var/lib/apt/lists/*
7
8 WORKDIR /app
```

Sau khi chạy lệnh `docker compose up -d` (tại thư mục chứa file `docker-compose.yml`) ta được container như sau

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
c04d2de598e6	ubuntu:20.04	"/bin/bash"	13 hours ago	Exited (0) 5 hours ago	0.0.0.0:8080->8080/tcp, :::8080->8080/tcp	http_server-server-1

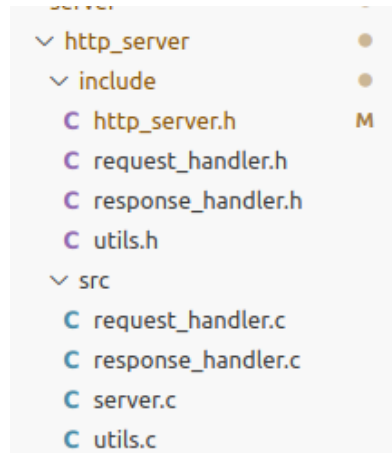
Tiến hành start container này và exec vào trong container. Folder mã nguồn được ánh xạ vào folder `app` của container này.

```
cocheche@ubuntu:~/Desktop/SP_TCP-IP$ sudo docker start c04d
c04d
cocheche@ubuntu:~/Desktop/SP_TCP-IP$ sudo exec -it c04d /bin/bash
sudo: exec: command not found
cocheche@ubuntu:~/Desktop/SP_TCP-IP$ sudo docker exec -it c04d /bin/bash
root@c04d2de598e6:/# ls
app bin boot dev etc home lib lib32 lib64 libx32 media mnt opt proc root run sbin srv sys tmp u
sr var
```

Khi vào trong container tiến hành cài thêm php-cgi để khởi động được Server

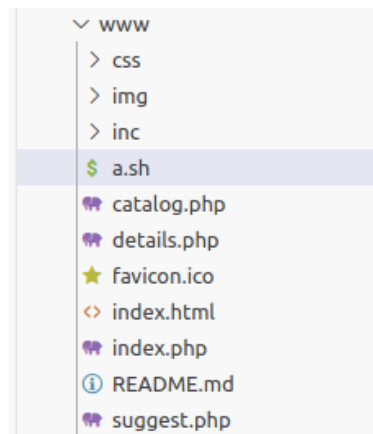
```
root@c04d2de598e6:/app# apt update && apt install php-cgi
Hit:1 http://archive.ubuntu.com/ubuntu focal InRelease
Get:2 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Get:3 http://archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Hit:4 http://archive.ubuntu.com/ubuntu focal-backports InRelease
Get:5 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages [4320 kB]
Get:6 http://archive.ubuntu.com/ubuntu focal-updates/universe amd64 Packages [1603 kB]
Get:7 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 Packages [4807 kB]
Get:8 http://security.ubuntu.com/ubuntu focal-security/universe amd64 Packages [1306 kB]
Fetched 12.3 MB in 1min 29s (138 kB/s)
Reading package lists... Done
Building dependency tree
Reading state information... Done
```

Tổ chức mã nguồn: Sắp xếp các tệp `.c` vào thư mục `src/` và các tệp `.h` vào thư mục `include/` (hoặc theo cấu trúc bạn đã dùng).



Tạo Thư mục Web Root:

- Tạo một thư mục tên là www (hoặc tên khác khớp với DOCUMENT_ROOT trong http_server.h) trong cùng thư mục chứa tệp thực thi Server.
- Đặt các tệp tĩnh vào đó: ví dụ index.html, style.css, image.jpg.
- Đặt các tệp PHP vào đó: ví dụ info.php (nội dung: `<?php phpinfo(); ?>`), hoặc một script xử lý form POST.



Tạo Makefile cho ứng dụng: Tạo một tệp Makefile để tự động hóa quá trình biên dịch. Đảm bảo liên kết với thư viện pthread (-lpthread). Không cần liên kết với thư viện SSL (-lssl -lcrypto).

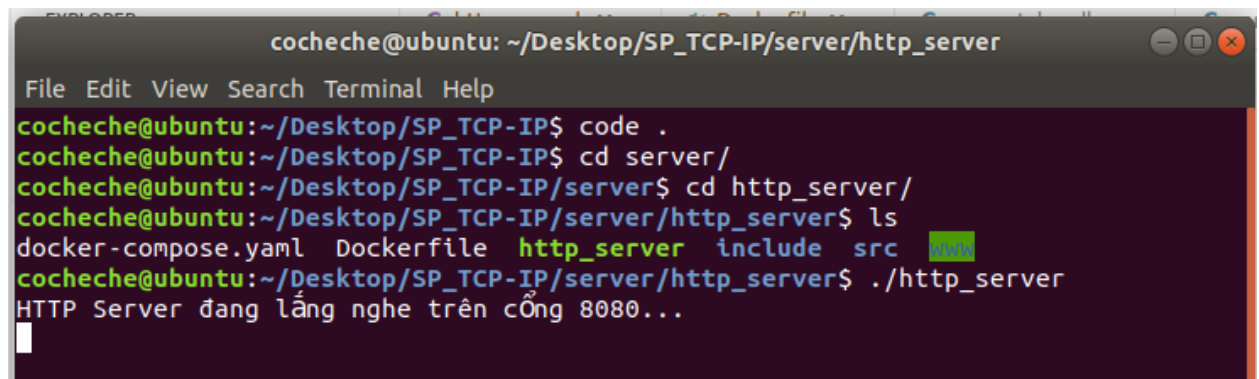
```
# MT_LDFLAGS = -pthread -lssh
HTTPS_LDFLAGS = -lpthread -lssl -lcrypto
|
```

```
## https server
$(HTTPS_SERVER_EXEC): $(HTTPS_SERVER_OBJ)
    $(CC) $(CFLAGS) -o $@ $^ $(HTTPS_LDFLAGS)
## end
```

Biên dịch chương trình thông qua Makefile: Mở terminal trong thư mục gốc của dự án và chạy lệnh: `make clean && make`. Nếu không có lỗi, một tệp thực thi tên `http_server` sẽ được tạo ra.

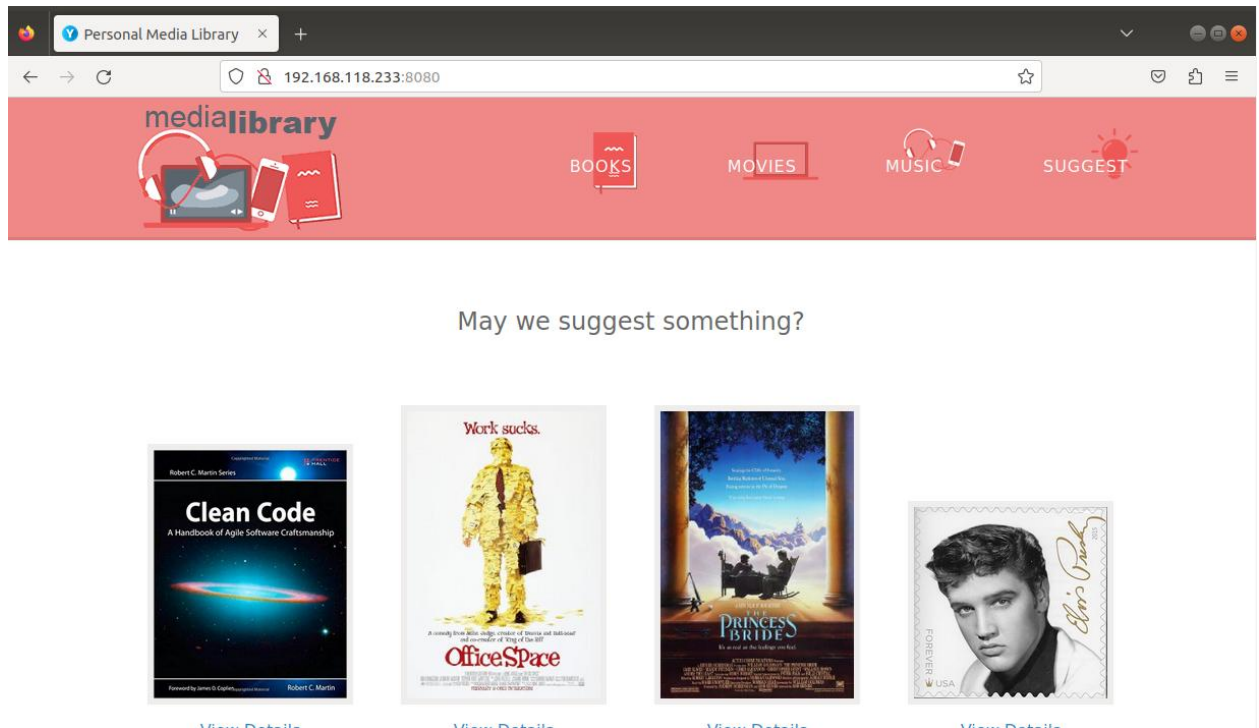


Chương trình `http_server`: Thực thi biên dịch: `./ http_server`. Server sẽ bắt đầu lắng nghe trên cổng `HTTP_PORT` (mặc định 8080).



Kiểm tra chương trình:

- Mở trình duyệt: Truy cập `http://localhost:8080` (hoặc IP của Server nếu chạy trên máy khác/container). Kiểm tra xem trang `index.html` (hoặc `index.php`) có hiển thị đúng không.



- Kiểm tra PHP (POST): Nếu có script PHP xử lý POST, có thể dùng CURL hoặc một trang HTML có form để gửi dữ liệu. Ví dụ: Ví dụ dùng curl để gửi dữ liệu 'name=Test&age=30' tới process.php

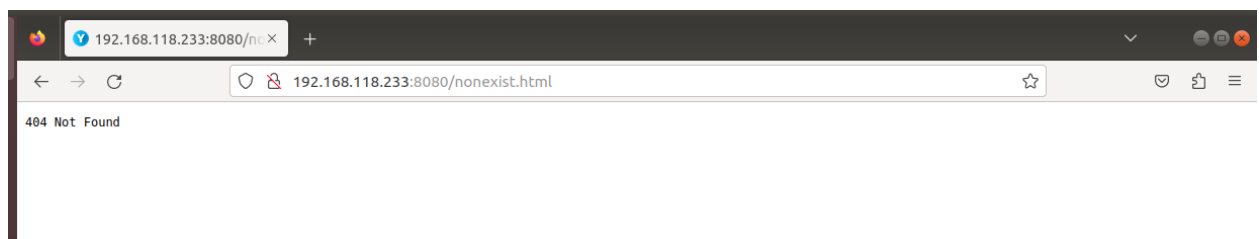
```

root@kali:~/avg/max/dev - 32.000/32.184/32.308/0.124 ms
cocheche@ubuntu:~$ curl -X POST -d "name=test&email=test@example.com&category=Books&title=Test" http://192.168.118.233:8080/suggest.php
cocheche@ubuntu:~$

```



Kiểm tra lỗi: Thử truy cập một tệp không tồn tại (<http://localhost:8080/nonexistent.html>) để xem phản hồi 404.



4.4.6. Kết luận

Việc xây dựng thành công máy chủ HTTP đa luồng đơn giản này đánh dấu một bước tiến quan trọng, không chỉ áp dụng kiến trúc xử lý đồng thời từ phần trước mà còn đi sâu vào tầng ứng dụng với giao thức HTTP. Qua quá trình này, nhóm đã thu được kinh nghiệm thực tế về:

- Cấu trúc và quy trình hoạt động của giao thức HTTP (Request-Response).
- Cách phân tích (parsing) yêu cầu HTTP để trích xuất thông tin quan trọng (method, path).
- Cách xử lý yêu cầu, ánh xạ URI tới tệp cục bộ và thực hiện các kiểm tra cơ bản.
- Cách xây dựng phản hồi HTTP hợp lệ, bao gồm dòng trạng thái và các header thiết yếu như Content-Type, Content-Length.
- Kỹ thuật xử lý tệp tĩnh và xác định loại MIME.

Tuy nhiên, máy chủ này vẫn còn nhiều hạn chế về tính năng, hiệu năng và bảo mật như đã phân tích. Nó chủ yếu phục vụ mục đích học tập và các ứng dụng đơn giản, yêu cầu tài nguyên thấp. Đây là nền tảng vững chắc để tiếp tục phát triển các tính năng nâng cao hơn như hỗ trợ HTTPS (mã hóa kết nối - sẽ được trình bày ở phần 4.5) giúp cải thiện bảo mật. Việc tích hợp (dù cơ bản) với CGI như php-cgi cũng mở ra hướng xử lý nội dung động, dù còn nhiều thách thức

4.5. HTTPS Server

4.5.1. Mô tả ứng dụng và hạn chế

Về cơ bản, máy chủ HTTPS này vẫn giữ nguyên kiến trúc đa luồng từ phiên bản trước, cho phép xử lý nhiều người dùng kết nối cùng lúc. Nó vẫn có khả năng trong việc phục vụ các tệp tĩnh quen thuộc (như trang HTML, tệp CSS định dạng, hay hình ảnh) và cũng có khả năng thực thi các script PHP-CGI để tạo ra nội dung web động theo yêu cầu.

Tuy nhiên, điểm khác biệt cốt lõi là mọi giao tiếp giờ đây đều diễn ra trên một kênh được mã hóa an toàn. Để hoạt động, máy chủ HTTPS yêu cầu hai thành phần thiết yếu: một tệp chứng chỉ số (.crt) và một tệp khóa riêng tư (.key). Những tệp này đóng vai trò như "chứng minh thư" và "chìa khóa bí mật" của Server, cho phép client xác thực Server và

thiết lập kết nối mã hóa. Nó sẽ lắng nghe các kết nối đến trên một cổng riêng biệt (thường là 9443 trong ví dụ này, khác với cổng 8080 của HTTP thông thường).

Khi một client kết nối, máy chủ và client sẽ thực hiện một 'bắt tay' SSL/TLS để thiết lập kênh mã hóa. Sau đó, máy chủ sẽ giải mã yêu cầu HTTP từ client, xử lý yêu cầu đó (ví dụ: tìm tệp tĩnh hoặc chạy script PHP), mã hóa phản hồi HTTP và gửi lại cho client qua kênh bảo mật đã thiết lập.

Mục đích chính của việc nâng cấp này là đảm bảo tính bí mật và toàn vẹn của dữ liệu trong quá trình truyền tải, bảo vệ thông tin nhạy cảm (như mật khẩu, dữ liệu cá nhân) khỏi nguy cơ bị nghe lén hay sửa đổi trên đường đi, mang lại một trải nghiệm web an toàn hơn cho người dùng. Có thể nói cách khác, máy chủ HTTPS này là phiên bản bảo mật của máy chủ HTTP ở phần 4.4.

Hạn chế của ứng dụng:

- Kế thừa hạn chế từ phần 4.4: Vẫn giữ các hạn chế về hiệu năng của PHP-CGI (tạo tiến trình mới cho mỗi request PHP) và khả năng mở rộng của mô hình Thread-Per-Client khi số lượng kết nối đồng thời quá lớn. Các tính năng HTTP nâng cao (Keep-Alive, Chunked Encoding, Compression) vẫn chưa được hỗ trợ.

- Yêu cầu quản lý chứng chỉ: Máy chủ phụ thuộc vào việc cung cấp các tệp chứng chỉ và khóa hợp lệ. Việc tạo, quản lý và gia hạn chứng chỉ (đặc biệt là chứng chỉ được tin cậy bởi trình duyệt) là một quy trình riêng biệt. Máy chủ này hoạt động tốt với chứng chỉ tự ký (self-signed) cho mục đích thử nghiệm, nhưng trình duyệt sẽ hiển thị cảnh báo bảo mật.

- Cấu hình SSL/TLS cơ bản: Việc triển khai chỉ sử dụng các cài đặt SSL/TLS mặc định của OpenSSL. Các cấu hình nâng cao như lựa chọn bộ mã hóa (cipher suite), phiên bản TLS cụ thể, Perfect Forward Secrecy (PFS), OCSP Stapling chưa được triển khai.

- Xử lý lỗi SSL: Việc xử lý lỗi trong quá trình thiết lập hoặc giao tiếp SSL còn cơ bản, chủ yếu dựa vào việc in lỗi ra stderr thông qua `ERR_print_errors_fp`.

4.5.2. Kiến trúc và các hàm cốt lõi

4.5.2.1 Kiến trúc

Vẫn giữ nguyên kiến trúc đa luồng từ Phần 4.4: Luồng chính (main thread) lắng nghe kết nối, chấp nhận kết nối mới và tạo một luồng xử lý riêng (worker thread) cho mỗi client. Tích hợp OpenSSL vào quá trình thiết lập Server và xử lý kết nối:

1. Khởi tạo Server (trong start_server và main):
 - a. Khởi tạo thư viện OpenSSL (SSL_library_init, OpenSSL_add_all_algorithms, SSL_load_error_strings).
 - b. Tạo ngữ cảnh SSL (SSL_CTX) bằng SSL_CTX_new với phương thức phù hợp (vd: SSLv23_server_method). SSL_CTX hoạt động như một "nhà máy" để tạo các đối tượng SSL cho từng kết nối.
 - c. Tải tệp chứng chỉ (.crt) và khóa riêng tư (.key) vào SSL_CTX bằng SSL_CTX_use_certificate_file và SSL_CTX_use_PrivateKey_file. Các tệp này được truyền qua tham số dòng lệnh (--ssl cert_file key_file).
 - d. Server socket được bind và listen trên cổng HTTPS (9443).
2. Xử lý Kết nối Mới (trong start_server):
 - a. Sau khi accept() một kết nối client (trả về client_fd), một đối tượng SSL mới được tạo từ SSL_CTX bằng SSL_new().
 - b. Đối tượng SSL này được gắn với file descriptor của client bằng SSL_set_fd(ssl, client_fd).
 - c. Cả client_fd và con trỏ ssl (cùng với cờ use_ssl) được đóng gói vào struct client_info và truyền cho luồng xử lý mới.
3. Xử lý Client (trong handle_client):
 - a. Luồng con nhận struct client_info.
 - b. Thực hiện quá trình SSL handshake bằng cách gọi SSL_accept(ssl). Hàm này sẽ xử lý việc trao đổi thông tin mã hóa với client.

- c. Sau khi `SSL_accept()` thành công: Tất cả các thao tác đọc/ghi dữ liệu với client phải được thực hiện thông qua các hàm OpenSSL tương ứng: `SSL_read(ssl, buffer, ...)` thay cho `read(client_fd, buffer, ...)` và `SSL_write(ssl, buffer, ...)` thay cho `write(client_fd, buffer, ...)`.
 - d. Hàm `handle_request` được gọi, truyền thêm con trỏ `ssl` và cờ `use_ssl`.
 - e. Khi kết thúc, cần giải phóng đối tượng SSL bằng `SSL_free(ssl)` trước khi đóng `client_fd`.
4. Xử lý Request/Response (trong `request_handler.c`, `response_handler.c`):
- a. Các hàm như `handle_request`, `handle_php_request`, `send_response` được sửa đổi để nhận thêm tham số SSL `*ssl` và `int use_ssl`.
 - b. Bên trong các hàm này, khi cần đọc hoặc ghi dữ liệu với client, chúng sẽ kiểm tra cờ `use_ssl`. Nếu là `true`, chúng sử dụng `SSL_read/SSL_write`; nếu là `false` (trường hợp dự phòng hoặc để tương thích ngược), chúng sử dụng `read/write` thông thường.

4.5.2.2 Các hàm cốt lõi

1. `server.c`:

- a. `main()`: Phân tích tham số dòng lệnh để xác định chế độ hoạt động (HTTP/HTTPS), lấy đường dẫn tệp chứng chỉ và khóa. Gọi `start_server`.
- b. `start_server(server_config config)`: Hàm chính khởi tạo Server. Thực hiện các bước tạo socket, bind, listen. Nếu `config.use_ssl` là `true`, gọi `create_ssl_context` và `load_certificates`. Trong vòng lặp `while(1)`, gọi `accept`, tạo đối tượng SSL nếu cần, tạo struct `client_info`, tạo và tách luồng `handle_client`.
- c. `handle_client(void *arg)`: Hàm thực thi trong mỗi luồng con. Ép kiểu `arg` thành `client_info*`. Thực hiện `SSL_accept` (nếu `use_ssl`). Đọc yêu cầu bằng `SSL_read` hoặc `read`. Gọi `handle_request`. Dọn dẹp (`SSL_free`, `close`, `free(info)`).

- d. `create_ssl_context()`: Khởi tạo OpenSSL và tạo `SSL_CTX`.
 - e. `load_certificates(SSL_CTX *ctx, const char *cert_file, const char *key_file)`: Tải chứng chỉ và khóa vào `SSL_CTX`.
2. `request_handler.c`:
- a. `handle_request(int client_fd, const char *request, SSL *ssl, int use_ssl)`: Hàm trung tâm xử lý request. Phân tích dòng đầu (method, path, protocol). Phân tích header (`parse_headers`), body (nếu POST/PUT). Xác định loại yêu cầu (tĩnh/PHP). Gọi `handle_php_request` hoặc xử lý file tĩnh. Sử dụng `ssl` và `use_ssl` khi gọi các hàm gửi/nhận hoặc `handle_php_request`.
 - b. `handle_php_request(int client_fd, ..., SSL *ssl, int use_ssl)`: Triển khai logic CGI cho PHP. Thiết lập biến môi trường. Tạo pipe, fork. Tiến trình con thực thi `php-cgi`. Tiến trình cha đọc output từ pipe và gửi lại cho client bằng `SSL_write` hoặc `write` (thông qua `send_response` hoặc trực tiếp nếu cần xử lý header/body phức tạp hơn như trong code).
 - c. `parse_headers()`, `set_cgi_headers()`: Các hàm tiện ích xử lý header HTTP và biến môi trường CGI.
3. `response_handler.c`:
- a. `send_response(int client_fd, ..., SSL *ssl, int use_ssl)`: Hàm gửi các phản hồi HTTP đơn giản. Kiểm tra `use_ssl` để quyết định dùng `SSL_write` hay `write`.
4. `utils.c`:
- a. `get_mime_type()`, `url_decode()`: Các hàm tiện ích không thay đổi.
5. Thư viện OpenSSL: Các hàm quan trọng được sử dụng bao gồm:
- a. `SSL_library_init`
 - b. `SSL_load_error_strings`
 - c. `SSLv23_server_method`
 - d. `SSL_CTX_new`
 - e. `SSL_CTX_use_certificate_file`

- f. SSL_CTX_use_PrivateKey_file
- g. SSL_new,
- h. SSL_set_fd, SSL_accept,
- i. SSL_read, SSL_write
- j. SSL_free
- k. SSL_CTX_free
- l. ERR_print_errors_fp

4.5.3. Các bước xây dựng ứng dụng

Chuẩn bị môi trường: Cài đặt trình biên dịch C (gcc), make, thư viện phát triển OpenSSL, php-cgi.

```

• cocheche@ubuntu:~/Desktop/SP_TCP-IP$ sudo apt install gcc && sudo apt install make
[sudo] password for cocheche:
Reading package lists... Done
Building dependency tree
Reading state information... Done
gcc is already the newest version (4:7.4.0-1ubuntu2.3).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
Reading package lists... Done
Building dependency tree
Reading state information... Done
make is already the newest version (4.1-9.1ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
○ cocheche@ubuntu:~/Desktop/SP_TCP-IP$

• cocheche@ubuntu:~/Desktop/SP_TCP-IP$ sudo apt install libssl-dev
Reading package lists... Done
Building dependency tree
Reading state information... Done
libssl-dev is already the newest version (1.1.1-1ubuntu2.1~18.04.23).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
○ cocheche@ubuntu:~/Desktop/SP_TCP-IP$

• cocheche@ubuntu:~/Desktop/SP_TCP-IP$ sudo apt install php-cgi
Reading package lists... Done
Building dependency tree
Reading state information... Done
php-cgi is already the newest version (1:7.2+60ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
○ cocheche@ubuntu:~/Desktop/SP_TCP-IP$

```

Tạo Chứng chỉ và Khóa (Self-Signed for Test): Mở terminal và chạy các lệnh sau để tạo khóa riêng tư (server.key) và chứng chỉ tự ký (server.crt) có hiệu lực 365 ngày:

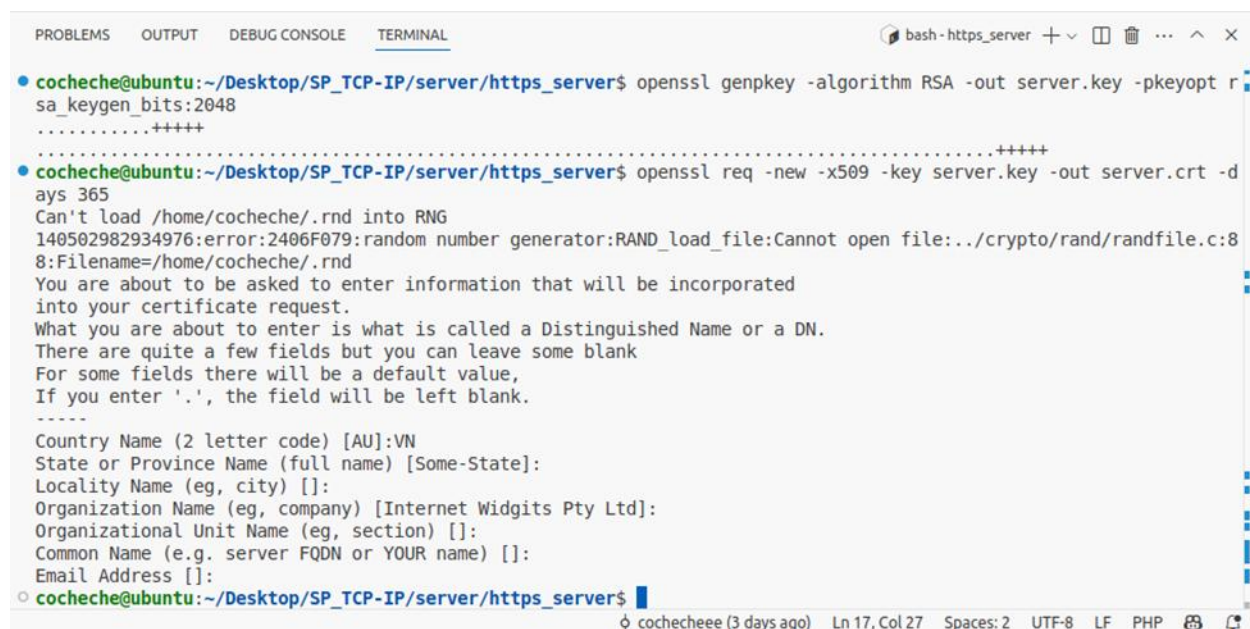
- Tạo khóa riêng tư RSA 2048 bit:

```
openssl genpkey -algorithm RSA -out server.key -pkeyopt  
rsa_keygen_bits:2048
```

- Tạo yêu cầu ký chứng chỉ (CSR) và tự ký nó để tạo chứng chỉ X.509:

```
openssl req -new -x509 -key server.key -out server.crt -days 365
```

Nhập thông tin cho chứng chỉ (Country Name, Organization Name, Common Name, etc.). Đối với Common Name, nên nhập localhost nếu bạn định kiểm tra trên máy cục bộ.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
bash - https_server + v [ ] [ ] ... ^ x
• cocheche@ubuntu:~/Desktop/SP_TCP-IP/server/https_server$ openssl genpkey -algorithm RSA -out server.key -pkeyopt r
sa_keygen_bits:2048
.....+++++
.....+++++
• cocheche@ubuntu:~/Desktop/SP_TCP-IP/server/https_server$ openssl req -new -x509 -key server.key -out server.crt -d
ays 365
Can't load /home/cocheche/.rnd into RNG
140502982934976:error:2406F079:random number generator:RAND_load_file:Cannot open file:../crypto/rand/randfile.c:8
8:Filename=/home/cocheche/.rnd
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:VN
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:
Email Address []:
• cocheche@ubuntu:~/Desktop/SP_TCP-IP/server/https_server$
```

Biên dịch chương trình:

- Mở terminal trong thư mục gốc của dự án (chứa Makefile). Sau đó chạy lệnh `make clean && make`. Kiểm tra xem Makefile có liên kết đúng với thư viện OpenSSL không (thường là thêm `-lssl -lcrypto` vào cờ liên kết `LDFLAGS`). Nếu không có lỗi, một tệp thực thi (vd: `https_server`) sẽ được tạo.

```

• cocheche@ubuntu:~/Desktop/SP_TCP-IP$ make clean
rm -f client/st_client/v1/tcp_client.o client/st_client/v1/socket_utils.o client/st_client/v2/tcp_client_2.o client/st_client/v2/socket_utils_2.o server/st_server/v1/tcp_server.o server/st_server/v1/socket_utils.o server/st_server/v2/tcp_server_2.o server/st_server/v2/socket_utils_2.o client/st_client/v1/st_client_v1 client/st_client/v2/st_client_v2 server/st_server/v1/st_server_v1 server/st_server/v2/st_server_v2 client/mt_client/client_utils.o client/mt_client/client.o server/mt_server/server_utils.o server/mt_server/server.o server/mt_server/client_handler.o client/mt_client/mt_client server/mt_server/mt_server server/http_server/http_server server/http_server/src/response_handler.o server/http_server/src/request_handler.o server/http_server/src/server.o server/http_server/src/utils.o server/https_server/https_server server/https_server/src/response_handler.o server/https_server/src/request_handler.o server/https_server/src/server.o server/https_server/src/utils.o
• cocheche@ubuntu:~/Desktop/SP_TCP-IP$ make
gcc -Wall -Wextra -Iinclude -O2 -c client/st_client/v1/tcp_client.c -o client/st_client/v1/tcp_client.o
gcc -Wall -Wextra -Iinclude -O2 -c client/st_client/v1/socket_utils.c -o client/st_client/v1/socket_utils.o
client/st_client/v1/socket_utils.c: In function 'communicate_with_server':
client/st_client/v1/socket_utils.c:43:9: warning: ignoring return value of 'fgets', declared with attribute warn_unused_result [-Wunused-result]
     fgets(buffer, BUFSIZE, stdin);
     ^~~~~

```

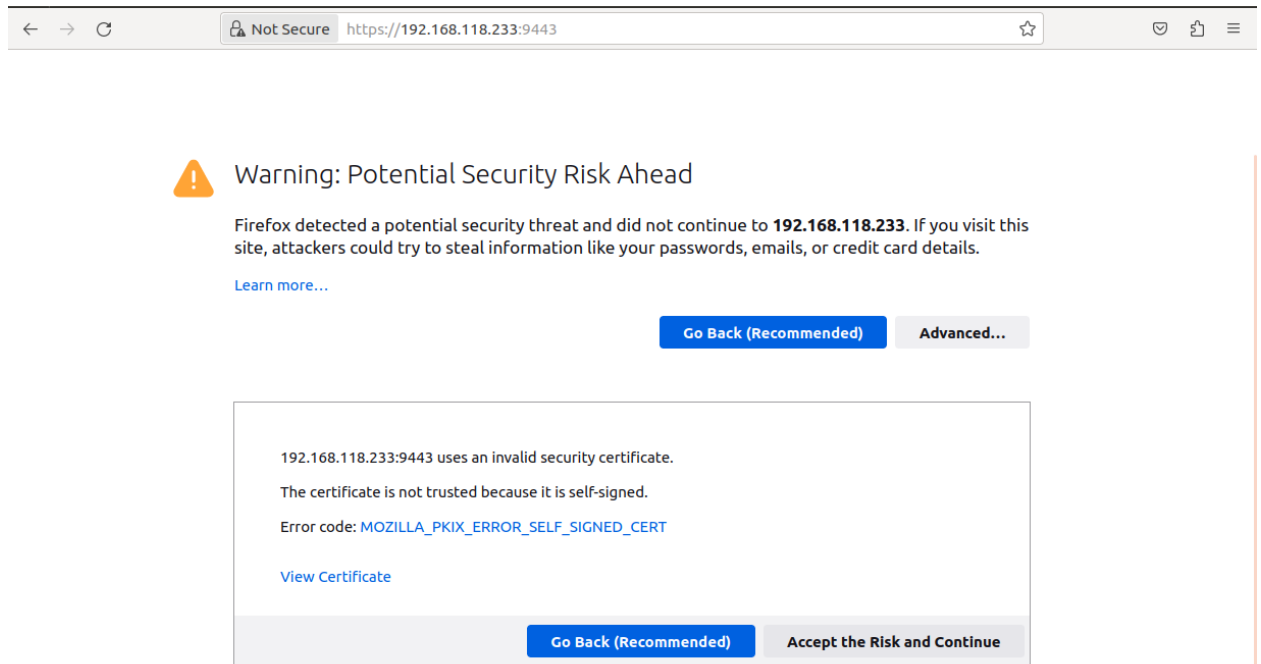
- Chạy Server: Chạy tệp thực thi với các tùy chọn SSL, trỏ đến tệp chứng chỉ và khóa bạn đã tạo: `./https_server --ssl ./server.crt ./server.key`
- Server sẽ bắt đầu lắng nghe trên cổng HTTPS_PORT (mặc định 9443).

```

cocheche@ubuntu:~/Desktop/SP_TCP-IP/server/https_server$ ./https_server --ssl server.crt server.key
HTTPS Server sẽ lắng nghe trên cổng 9443
Server đang lắng nghe...

```

Kiểm tra: Mở trình duyệt web: Truy cập `https://localhost:9443` (hoặc địa chỉ IP của Server nếu chạy trên máy khác). Sẽ có một cảnh báo bảo mật, trình duyệt sẽ hiển thị cảnh báo vì chứng chỉ là tự ký và không được tin cậy. Bạn cần chấp nhận rủi ro (tìm tùy chọn như "Advanced", "Proceed to localhost (unsafe)") để tiếp tục. Đây là phản ứng bảo mật chuẩn của trình duyệt đối với chứng chỉ không được ký bởi Certificate Authority (CA) đáng tin cậy.



Khi cấu hình thành công đường dẫn sẽ thành <https://<IP-server>:<port>/>



Sau đó kiểm tra hiển thị như phần 4.4.

4.5.4. Kết luận

Việc triển khai thành công máy chủ HTTPS đánh dấu một bước tiến quan trọng và cần thiết, kế thừa và bảo mật hóa nền tảng máy chủ HTTP đa luồng đã xây dựng ở phần 4.4. Mục tiêu chính của phần này là tích hợp lớp mã hóa SSL/TLS thông qua thư viện OpenSSL, và mục tiêu này đã được hoàn thành.

Giờ đây, mọi giao tiếp giữa client và Server đều được bảo vệ bằng mã hóa, đảm bảo tính bí mật và toàn vẹn cho dữ liệu truyền đi, đặc biệt quan trọng đối với các thông tin nhạy cảm. Quá trình này không chỉ củng cố kiến thức về cách thức hoạt động của giao thức HTTPS mà còn mang lại kinh nghiệm thực tế trong việc làm việc với thư viện OpenSSL, xử lý chứng chỉ số và khóa riêng tư trong một ứng dụng mạng viết bằng C. Máy chủ vẫn

duy trì khả năng phục vụ tệp tĩnh và thực thi PHP-CGI, nhưng giờ đây thực hiện các tác vụ đó qua một kênh liên lạc an toàn.

Tuy nhiên, cần nhìn nhận rằng phiên bản này vẫn kế thừa những hạn chế về hiệu năng của mô hình đa luồng (thread-per-client) và cơ chế PHP-CGI đã tồn tại từ phần 4.4. Đồng thời, việc triển khai SSL/TLS hiện tại còn ở mức cơ bản, sử dụng các cấu hình mặc định và yêu cầu quản lý chứng chỉ (với chứng chỉ tự ký gây cảnh báo trình duyệt).

Hướng phát triển tiềm năng trong tương lai bao gồm việc tối ưu hóa hiệu năng (ví dụ: chuyển sang mô hình non-blocking I/O hoặc thread pool, sử dụng FastCGI thay cho PHP-CGI), tinh chỉnh cấu hình SSL/TLS nâng cao (lựa chọn bộ mã hóa mạnh, phiên bản TLS mới nhất), và tích hợp cơ chế xử lý và xác thực chứng chỉ một cách linh hoạt hơn.

Tóm lại, việc hoàn thành máy chủ HTTPS này không chỉ là một bài tập kỹ thuật mà còn là một bước đi thiết yếu hướng tới việc xây dựng các ứng dụng web thực tế, nơi yếu tố bảo mật luôn được đặt lên hàng đầu. Nó tạo dựng một nền tảng vững chắc cho việc phát triển các dịch vụ web phức tạp và an toàn hơn trong tương lai.

PHẦN KẾT LUẬN

Qua quá trình thực hiện đồ án, với trọng tâm được trình bày chi tiết trong các nội dung trên, nhóm chúng em đã tiến hành thiết kế, xây dựng và kiểm nghiệm một chuỗi các mô hình máy chủ mạng từ đơn giản đến phức tạp bằng ngôn ngữ lập trình C. Quá trình này bao gồm việc triển khai máy chủ TCP đơn luồng, máy chủ TCP đa luồng, máy chủ HTTP cơ bản với khả năng phục vụ tệp tĩnh và thực thi PHP-CGI, và cuối cùng là nâng cấp lên máy chủ HTTPS với mã hóa SSL/TLS. Môi trường kiểm thử Docker đã được sử dụng hiệu quả xuyên suốt quá trình để đảm bảo tính nhất quán và tối ưu hóa tài nguyên.

1. Kết quả đạt được

- **Nắm vững lập trình Socket cơ bản:** Đã xây dựng thành công các máy chủ TCP, qua đó hiểu rõ và vận dụng được các hàm API Socket cốt lõi (socket, bind, listen, accept, connect, read/recv, write/send, close) và các cấu trúc dữ liệu liên quan trong môi trường mạng TCP/IP.
- **Hiểu và giải quyết vấn đề đồng thời (concurrency):** Nhận diện được hạn chế blocking của mô hình đơn luồng và triển khai thành công kỹ thuật đa luồng (sử dụng pthread) để cho phép máy chủ xử lý đồng thời nhiều client, cải thiện đáng kể khả năng đáp ứng và hiệu năng so với mô hình ban đầu.
- **Xây dựng HTTP Server:** Phát triển được một máy chủ HTTP có khả năng phân tích các yêu cầu HTTP cơ bản (GET/POST), phục vụ các loại tệp tĩnh khác nhau dựa trên MIME type, và thực thi các script PHP thông qua cơ chế CGI, cung cấp nội dung động.
- **Triển khai Bảo mật HTTPS:** Tích hợp thành công thư viện OpenSSL vào máy chủ HTTP, thiết lập được kênh giao tiếp mã hóa SSL/TLS, hiểu được quy trình handshake, vai trò của chứng chỉ số và khóa riêng tư, đảm bảo tính bí mật và toàn vẹn cho dữ liệu truyền tải.
- **Kinh nghiệm thực tiễn với công cụ:** Tích lũy kinh nghiệm sử dụng ngôn ngữ C cho lập trình mạng cấp thấp và khai thác hiệu quả Docker như một công cụ mạnh mẽ cho việc thiết lập môi trường và kiểm thử ứng dụng mạng.

2. Ưu điểm của đề tài

- Tính thực tiễn và ứng dụng cao: Đề tài tập trung vào việc xây dựng các thành phần mạng cơ bản, mang lại kiến thức và kỹ năng lập trình C, mạng trực tiếp có thể áp dụng.
- Tiếp cận hệ thống: Việc phát triển đi từ mô hình đơn giản đến phức tạp giúp củng cố kiến thức nền tảng và làm nổi bật sự cần thiết, ưu nhược điểm của từng kỹ thuật (đơn luồng vs đa luồng, HTTP vs HTTPS, CGI).
- Sử dụng công cụ hiện đại: Việc tích hợp Docker vào quy trình kiểm thử giúp làm quen và thực hành với các công nghệ đang được sử dụng rộng rãi trong phát triển và vận hành phần mềm.

3. Nhược điểm và hạn chế

- Hiệu năng chưa tối ưu: Mô hình thread-per-client trở nên kém hiệu quả và tốn tài nguyên khi số lượng kết nối lớn. Cơ chế PHP-CGI với việc tạo tiến trình mới cho mỗi yêu cầu cũng là một điểm nghẽn hiệu năng đáng kể.
- Tính năng hạn chế: Các máy chủ được xây dựng chỉ hỗ trợ một tập con các tính năng của giao thức HTTP/1.1 (thiếu Keep-Alive, Chunked Encoding, Compression, Caching...) và cấu hình SSL/TLS còn ở mức cơ bản.
- Bảo mật cần cải thiện: Ngoài việc triển khai HTTPS cơ bản, các khía cạnh bảo mật tầng ứng dụng (như chống Path Traversal, xác thực đầu vào) chưa được xử lý triệt để. Việc sử dụng chứng chỉ tự ký cũng hạn chế việc kiểm thử trong môi trường giống production.
- Xử lý lỗi và Logging còn đơn giản: Cơ chế báo lỗi và ghi log còn sơ sài, gây khó khăn cho việc gỡ lỗi và giám sát hoạt động của máy chủ.
- Thiếu linh hoạt trong cấu hình: Nhiều tham số quan trọng (như đường dẫn gốc, cổng, kích thước buffer) được định nghĩa cứng trong mã nguồn.

4. Hướng phát triển của đề tài

Dựa trên những kết quả đạt được và các hạn chế đã nhận diện, đề tài có thể được tiếp tục phát triển theo các hướng sau để hoàn thiện và nâng cao tính ứng dụng.

- Tối ưu hóa Hiệu năng và Khả năng mở rộng:
 - Nghiên cứu và triển khai mô hình Non-blocking I/O sử dụng các cơ chế như epoll (trên Linux), kqueue (trên BSD/macOS) hoặc select/poll để xử lý hiệu quả hàng nghìn kết nối đồng thời chỉ với một số ít luồng.
 - Thay thế mô hình thread-per-client bằng Thread Pool để giảm chi phí tạo/hủy và quản lý luồng.
 - Tích hợp với các giải pháp thực thi PHP hiệu quả hơn như FastCGI (thông qua thư viện như spawn-fcgi và giao tiếp với PHP-FPM).
 - Mở rộng Tính năng Giao thức:
- Bổ sung hỗ trợ HTTP Keep-Alive để tái sử dụng kết nối TCP.
 - Triển khai Chunked Transfer Encoding cho các phản hồi động.
 - Tích hợp cơ chế Compression (gzip).
 - Nâng cao cấu hình SSL/TLS (lựa chọn cipher suite, phiên bản TLS, Perfect Forward Secrecy).
- Tăng cường Bảo mật và Ổn định:
 - Xây dựng cơ chế lọc và kiểm tra chặt chẽ các dữ liệu đầu vào từ client (URI, headers, body).
 - Phát triển hệ thống logging chi tiết và linh hoạt hơn.
 - Tích hợp cơ chế sử dụng/quản lý chứng chỉ SSL/TLS hợp lệ.
- Cải thiện tính linh hoạt: Cho phép cấu hình các tham số hoạt động của Server thông qua file cấu hình thay vì hardcoded.

Tóm lại, đề tài đã thành công trong việc xây dựng và phân tích các mô hình máy chủ mạng nền tảng bằng ngôn ngữ C. Những kiến thức và kinh nghiệm thực tế thu được từ quá trình này là vô cùng quý báu, tạo tiền đề vững chắc cho việc nghiên cứu sâu hơn và phát triển các hệ thống mạng phức tạp, hiệu quả và an toàn hơn trong tương lai.

5. Tài liệu tham khảo

1. Chiến Nguyễn (2024). “Giao thức TCP/IP là gì? Tổng hợp các kiến thức từ cơ bản đến nâng cao về mô hình này”. <https://fptshop.com.vn/tin-tuc/danh-gia/giao-thuc-tcp-ip-la-gi-171251>
2. OmniSecu.com. “TCP/IP: Free TCP/IP online course, Tutorials, Study materials, Guides, documentation”. <https://www.omniseku.com/tcpip/>
3. TechTarget. (n.d.). “What is TCP/IP?”. Kinza Yasar – Technical Writer, Mary E. Shacklett – Transworld Data, Amy Novotny – Senior Managing Editor. <https://www.techtarget.com/searchnetworking/definition/TCP-IP>
4. T3H. (2024). “Mô hình TCP/IP và những điều bạn nên biết”. <https://t3h.com.vn/tin-tuc/mo-hinh-tcpip-va-nhung-dieu-ban-nen-biet>
5. Thuận Nguyễn (2025). “Chống giao thức TCP/IP”. <https://viblo.asia/p/chong-giao-thuc-tcpip-EvbLbAavJnk>
6. Colin Perkins (2010). “Networked Systems”. <https://csparks.org/teaching/2009-2010/networked-systems/>
7. Jeffrey Vũ (2023). “How I Built a Simple HTTP Server from Scratch using C”. <https://dev.to/jeffreycoder/how-i-built-a-simple-http-server-from-scratch-using-c-739>
8. Shah, A. (2018). Hands-On network programming with C: Learn socket programming in C and write secure and optimized network code. Packt Publishing.